

Digital Logic Circuits

Timing Analysis

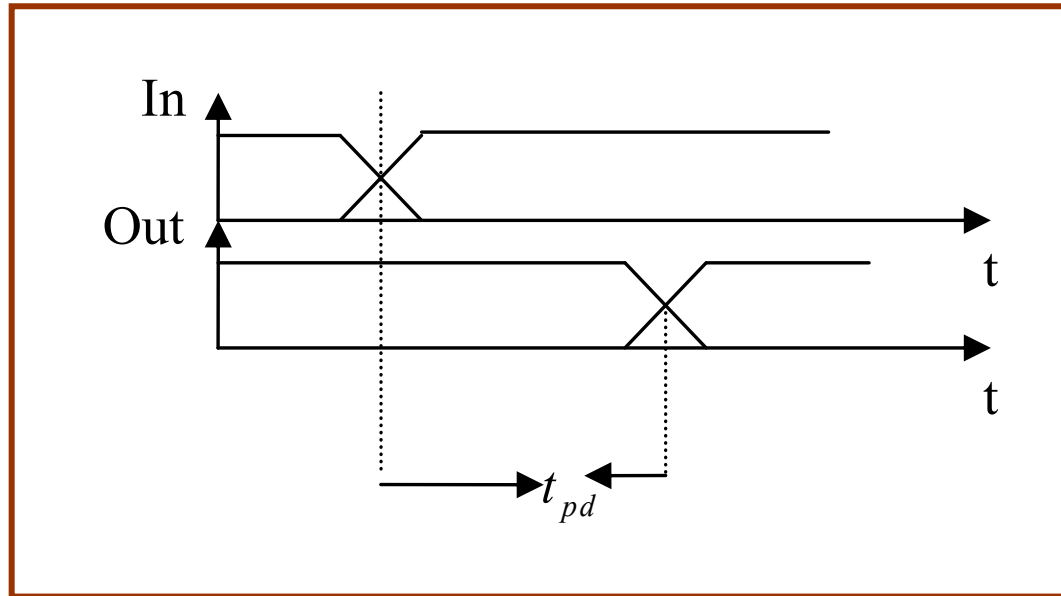
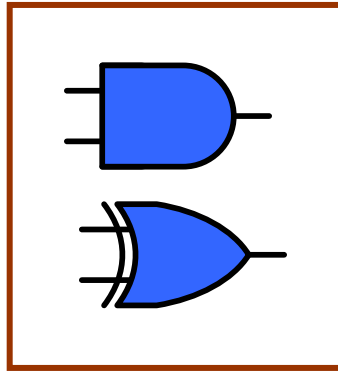
©Hanan Ribo

Combinational Circuits

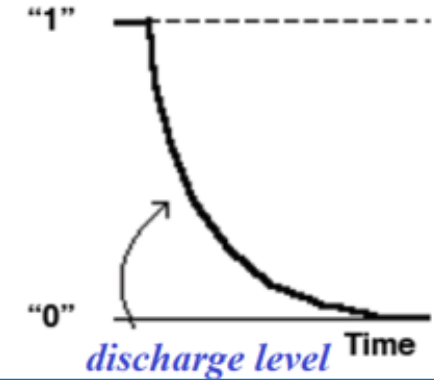
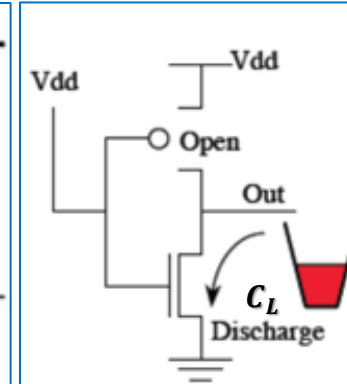
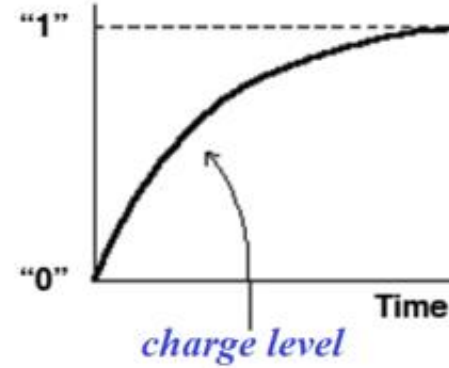
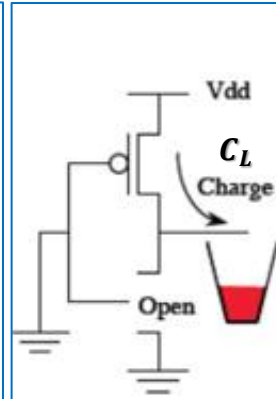
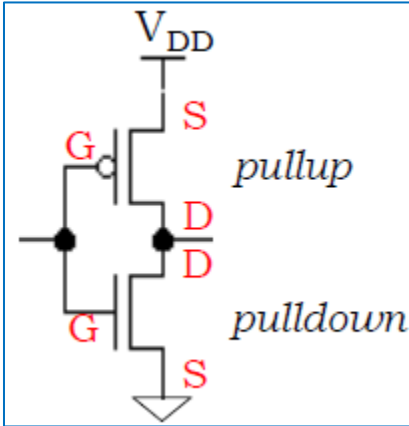
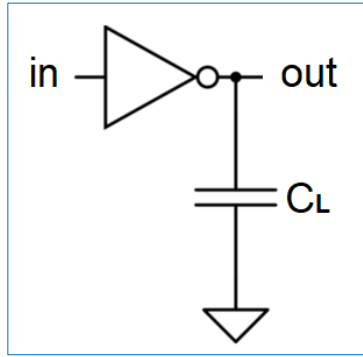
Timing Analysis

Logic Gates Timing

t_{pd} : Time from state change at input to state change at output



CMOS NOT Gate VTC (Voltage Transfer Curve)



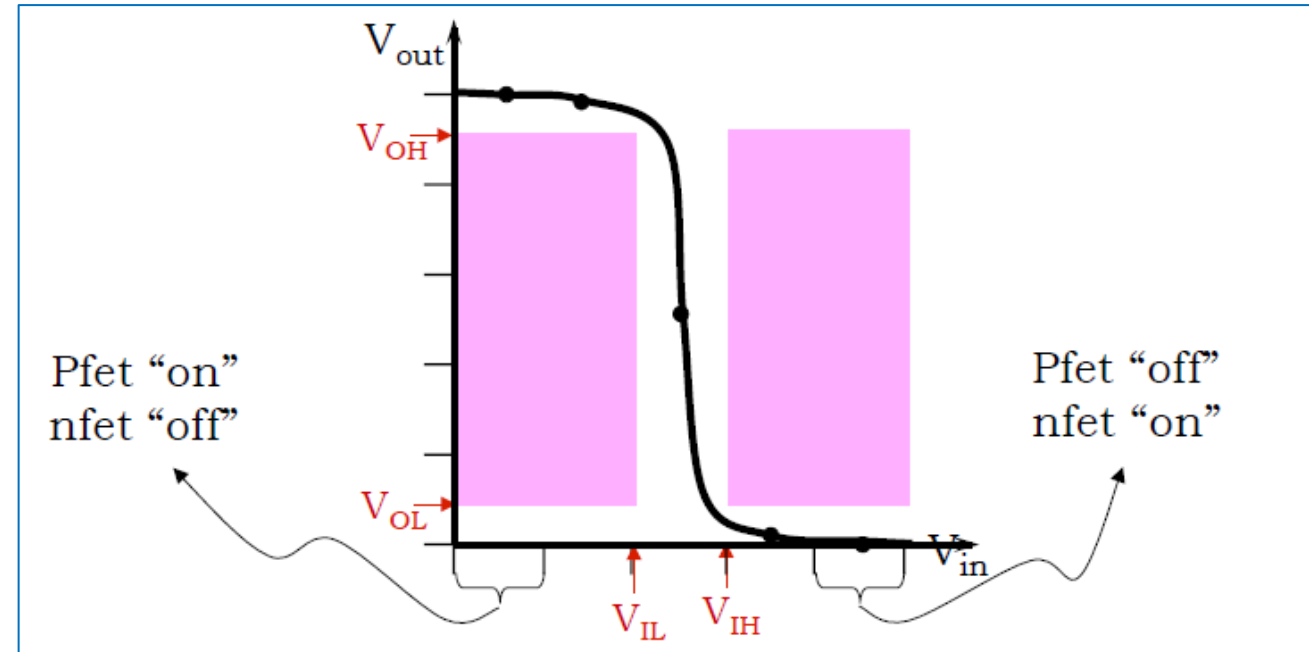
$$C_L = C_{out} + C_{line} + N \cdot C_{in}$$

C_{out} - parasitic capacitance due to transistors configuration between output and GND

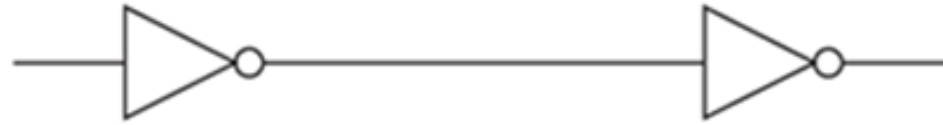
C_{line} - parasitic capacitance due to lines connection

$N \cdot C_{in}$ - input capacitance of N gates at the next rank only (CMOS), connected to its output (affects the Fanout)

$$t_{pd} = f(C_L, T[^\circ\text{C}], V_{CC})$$



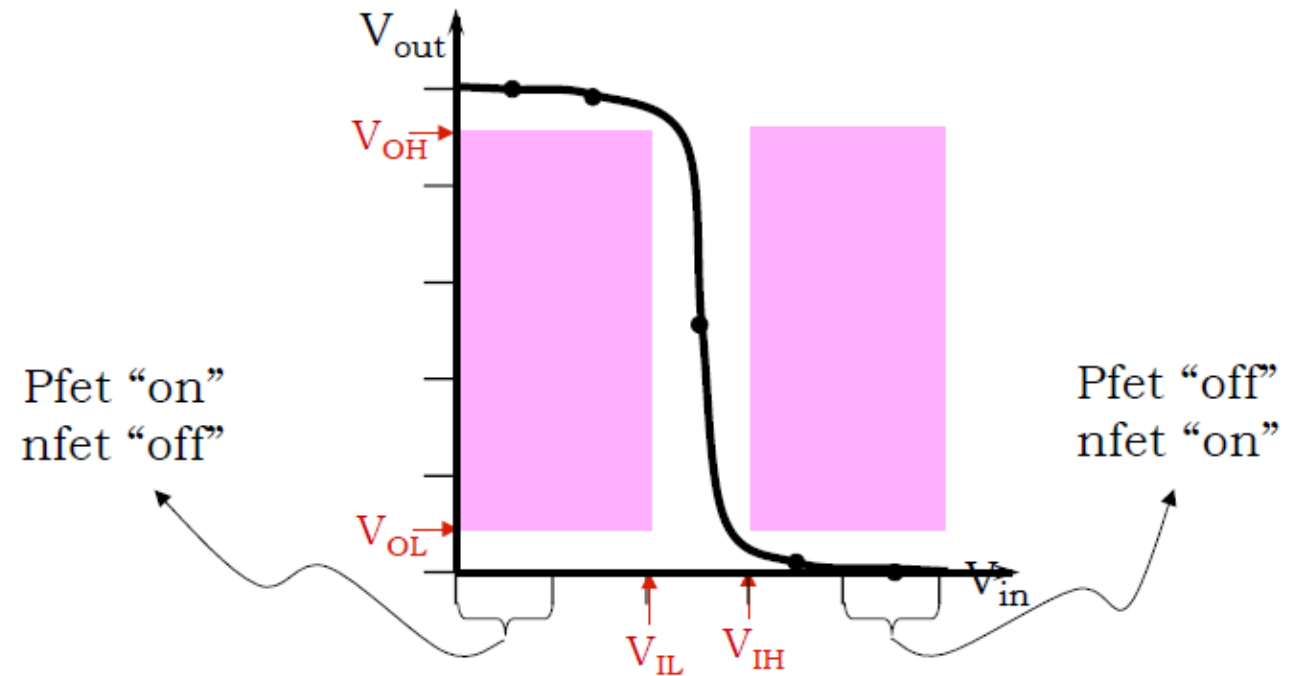
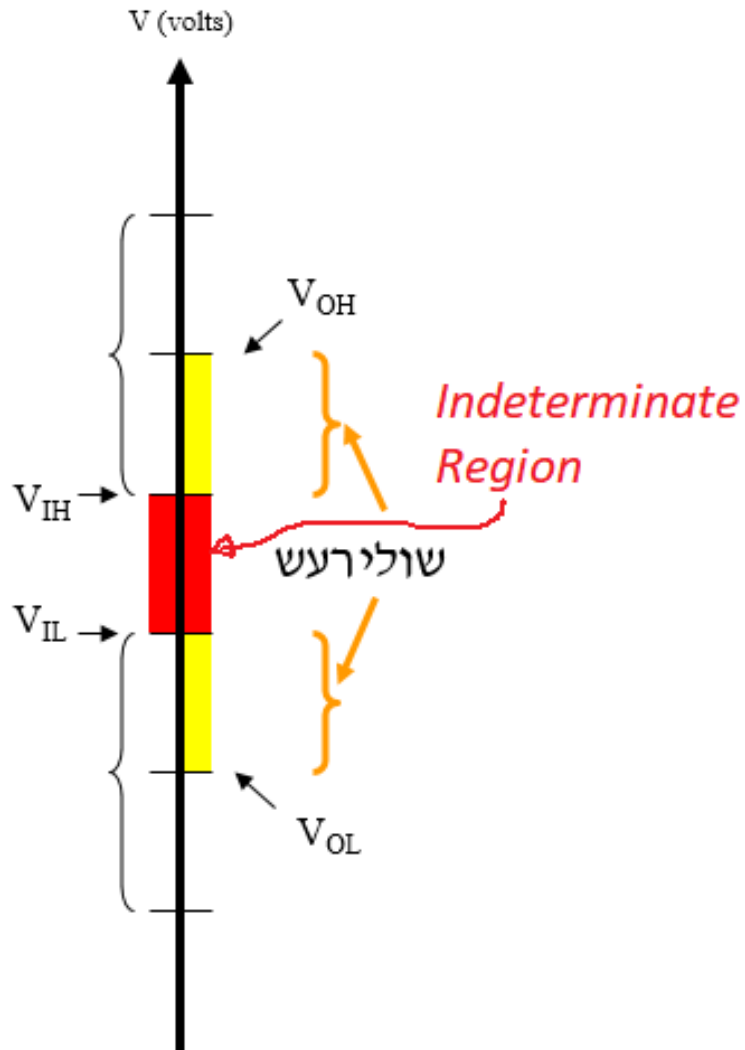
Digital Logic Abstraction



Output Characteristics

Input Characteristics

The noise margins overcome the noise that is added to the signal in the transition from one component to another

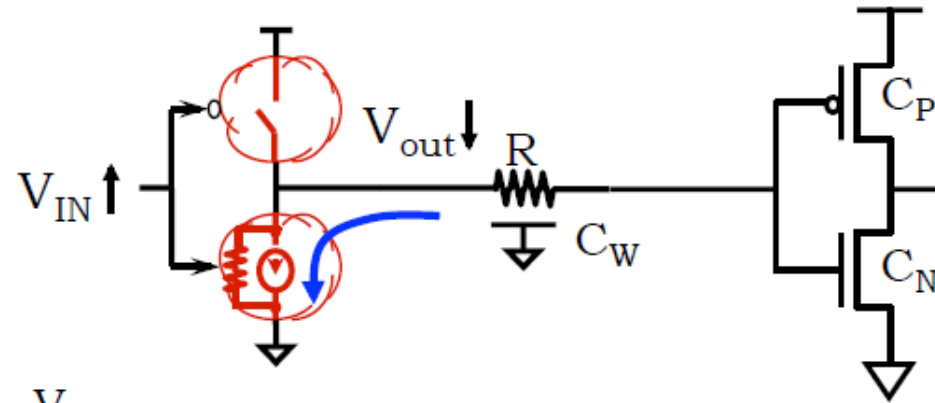


CMOS Timing Specifications

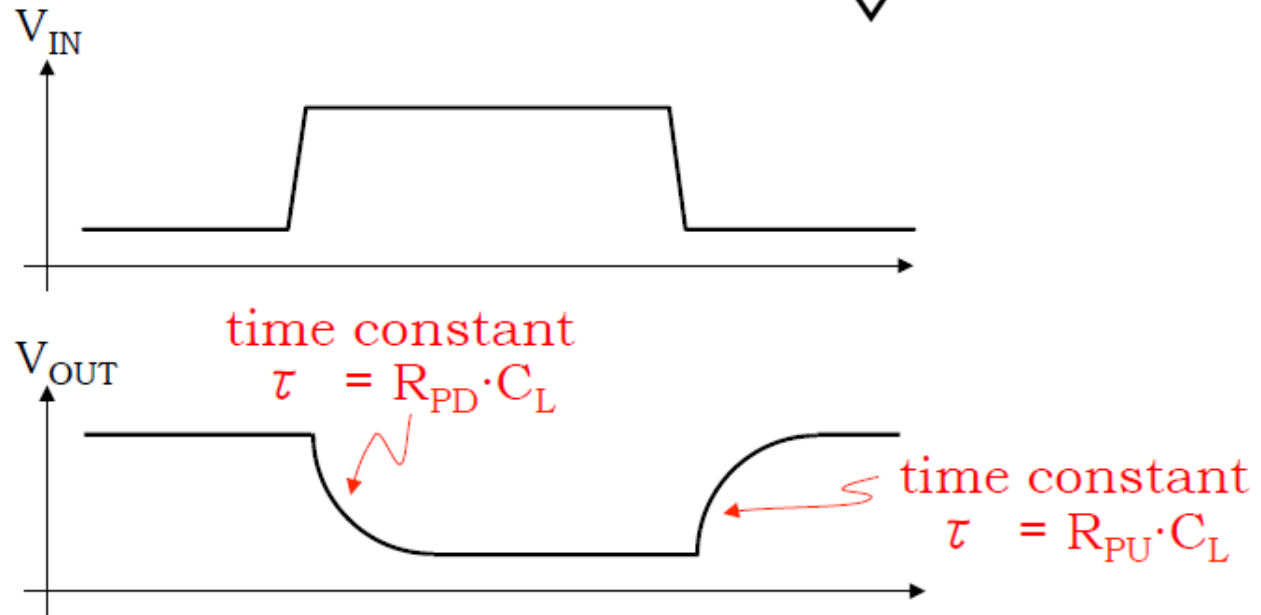
Circuit:



Electrical model:

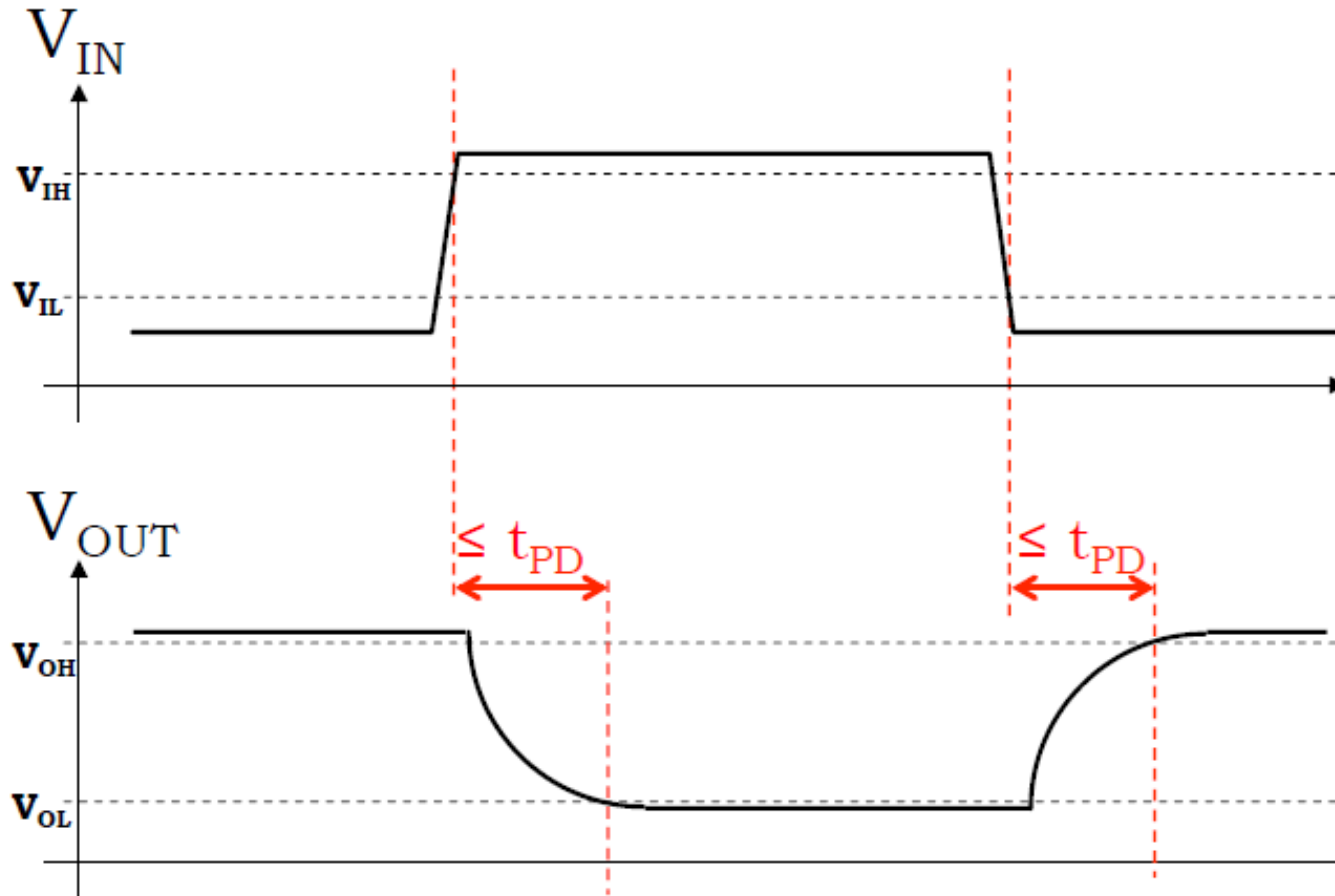


Waveforms:



Propagation delay (t_{PD}):

Propagation delay (t_{PD}): An UPPER BOUND on the delay from **valid inputs** to **valid outputs**.

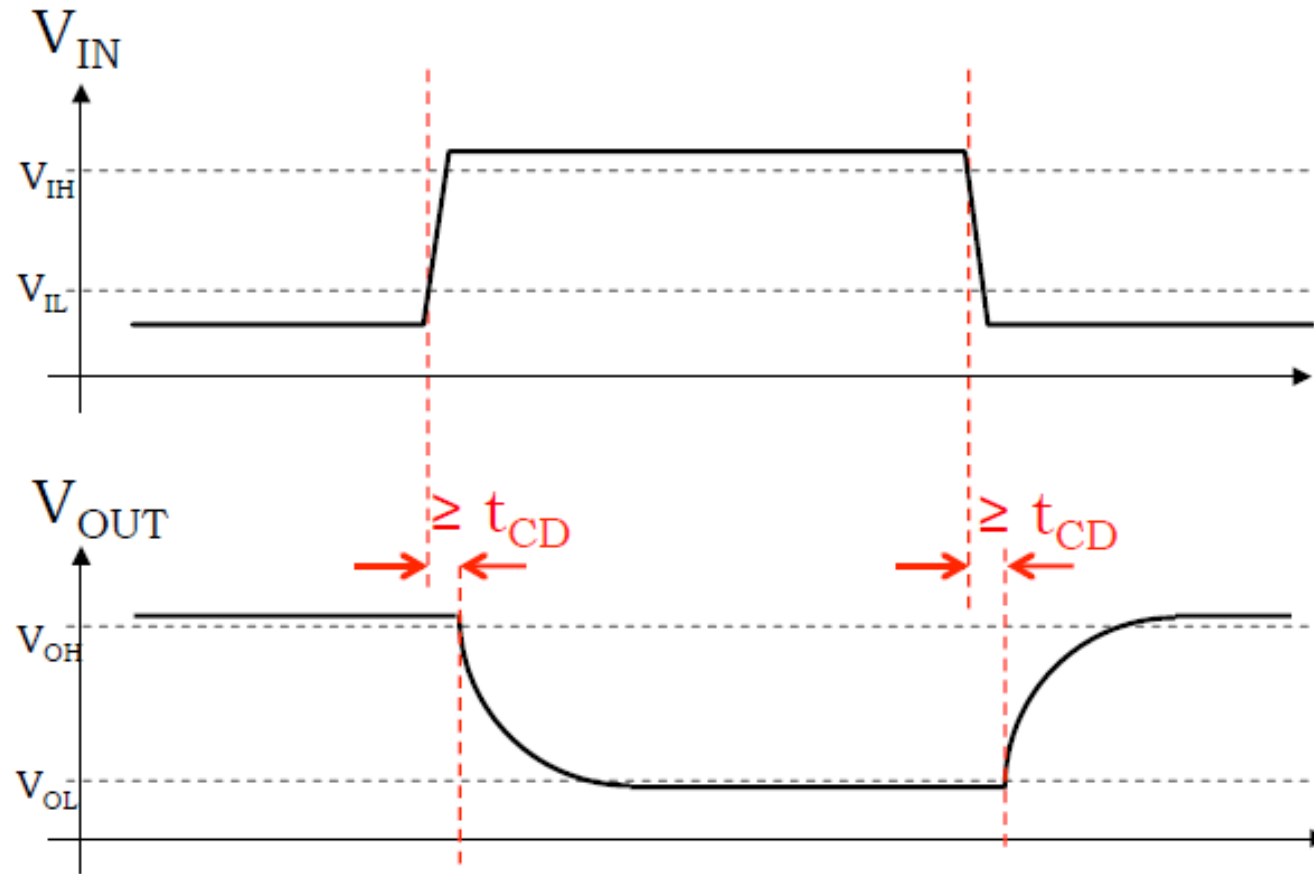


GOAL:
minimize
propagation
delay!

ISSUE:
keep
capacitances
low and
transistors
fast

Contamination Delay

Contamination delay (t_{CD}): A LOWER BOUND on the delay from any **invalid input** to an **invalid output**

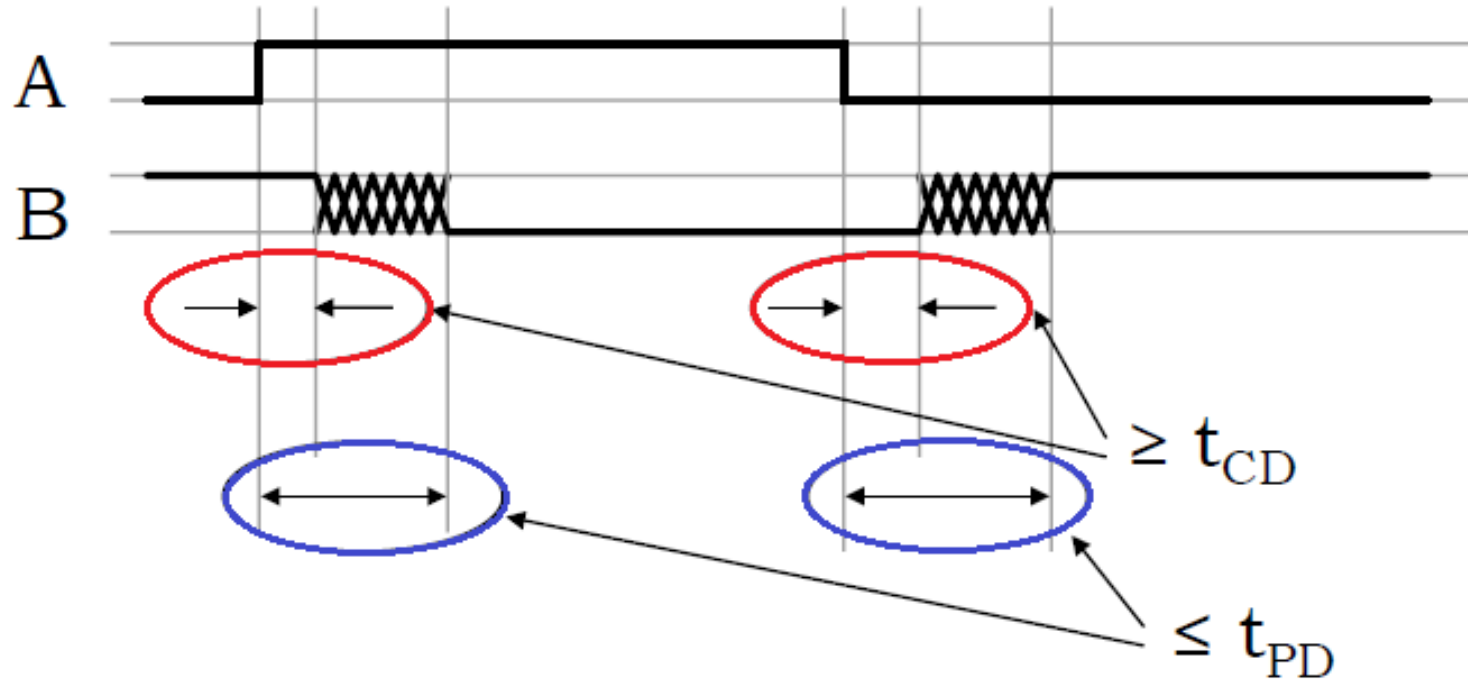
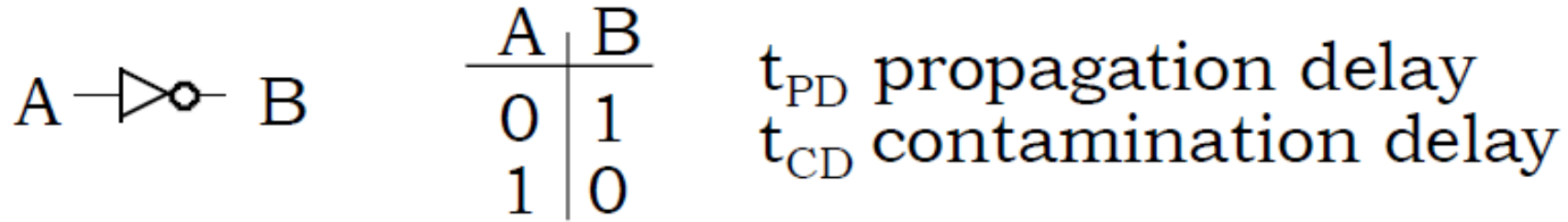


Do we really need t_{CD} ?

Usually not... it'll be important when we design circuits with registers (coming soon!)

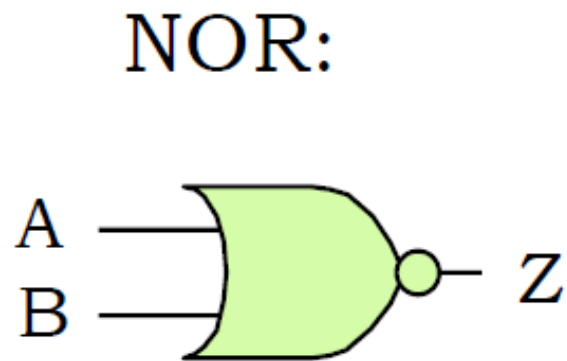
If t_{CD} is not specified, safe to assume it's 0.

Combinational Example

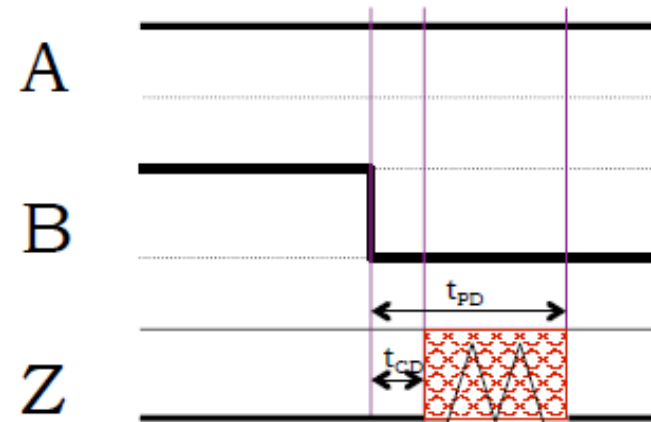


Static Discipline Obedience

- Definition: guarantee on logical elements that *"if inputs meet valid input thresholds, then the system guarantees outputs will meet valid output thresholds"*, named by Stephen A. Ward and Robert H. Halstead in 1990.
- Implication: Output guaranteed to be valid when *all* inputs have been valid for at least t_{PD} , and, outputs may become invalid no earlier than t_{CD} after an input changes!

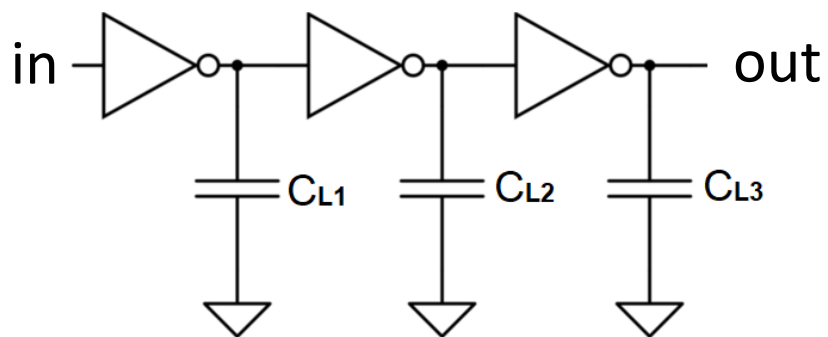
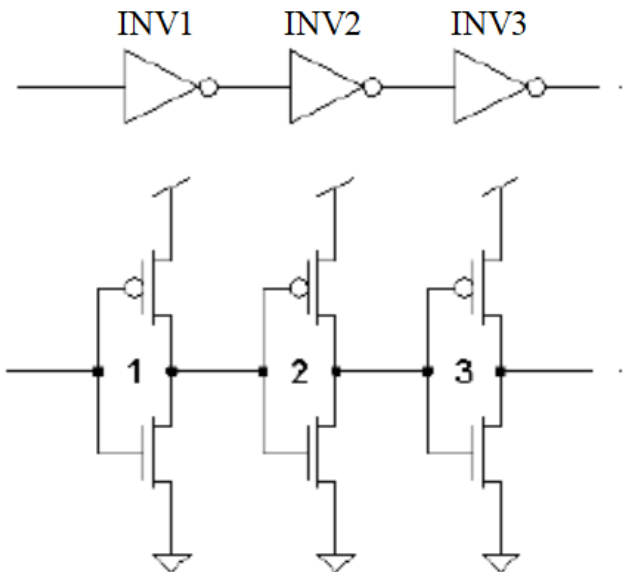


A	B	Z
0	0	1
0	1	0
1	0	0
1	1	0

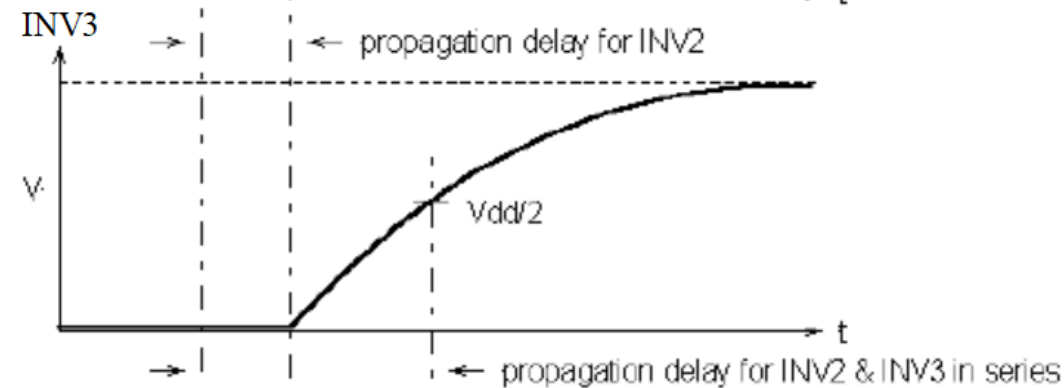
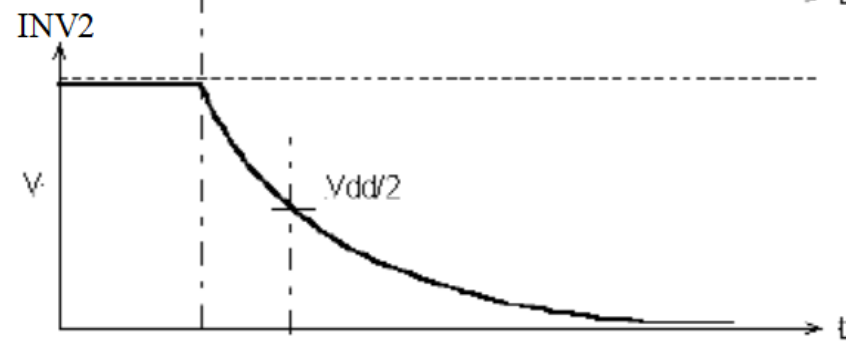
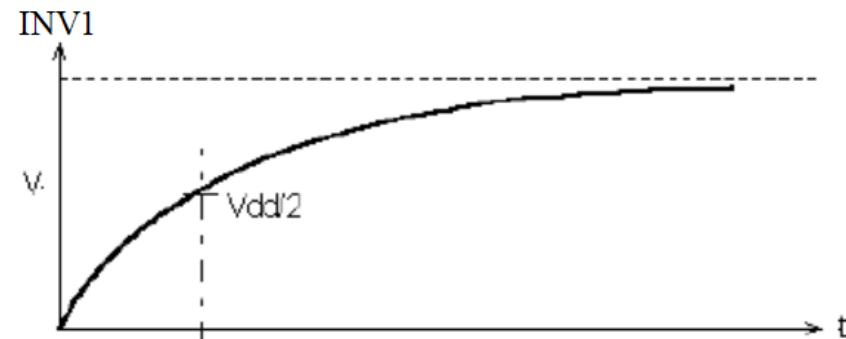
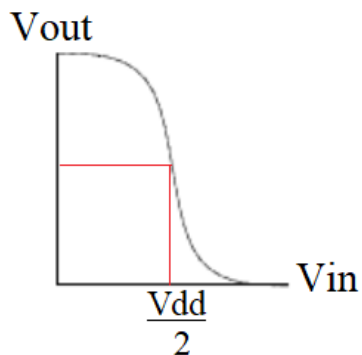


Cascaded gates delay

Cascaded gates



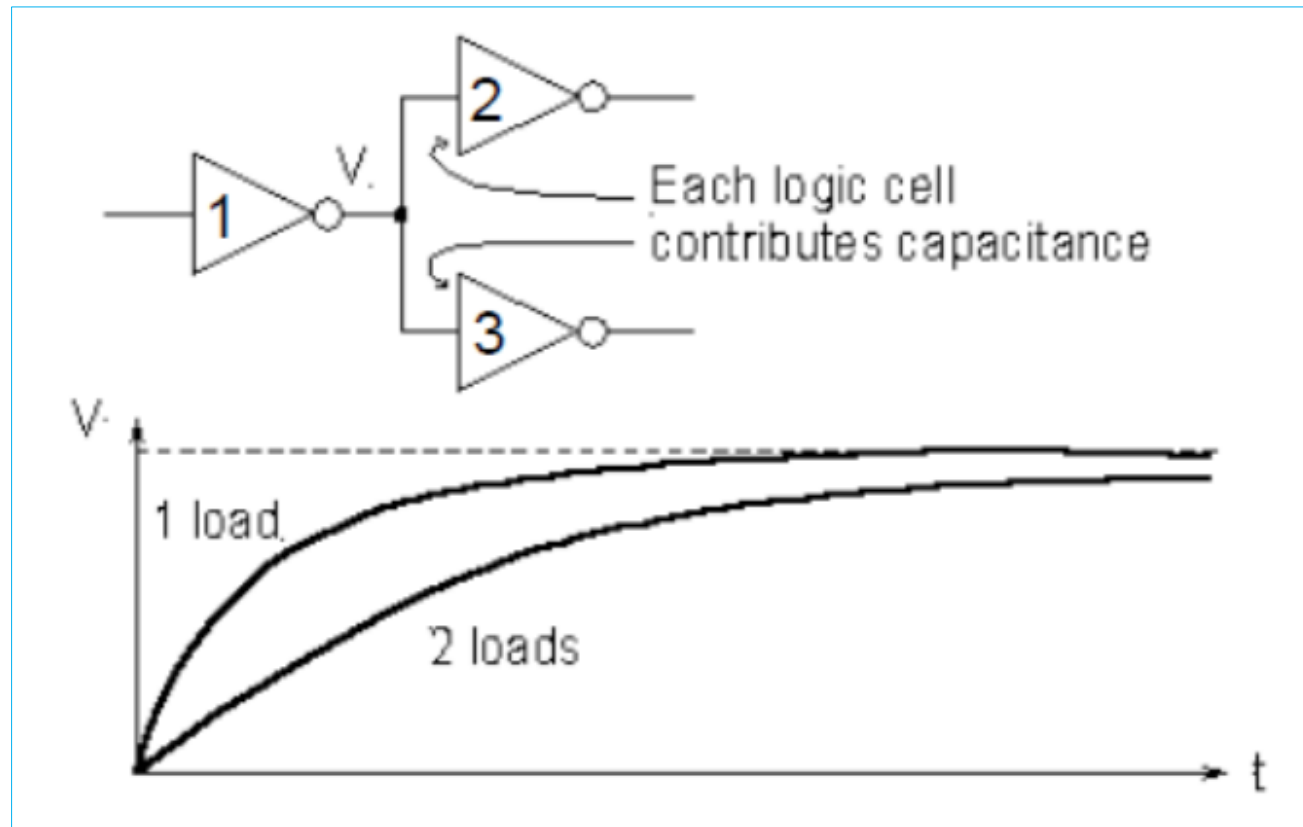
$$t_{pd} = t_{pd1} + t_{pd2} + t_{pd3}$$



$$T_{pd} = T_{pd1} + T_{pd2} + T_{pd3}$$

Fan-out delay

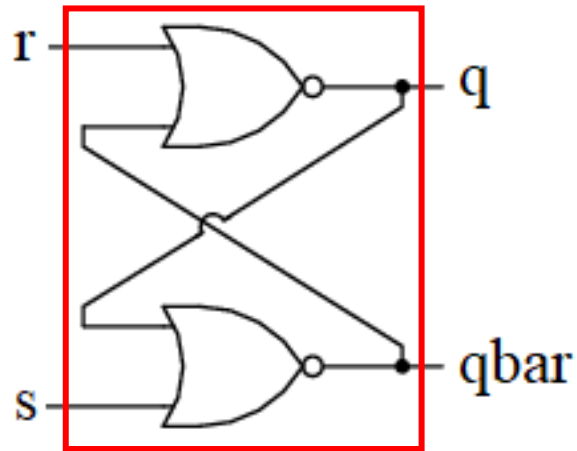
- The delay of a gate is proportional to its output capacitance. Connecting the output of a gate to more than one other gate increases its output capacitance.
- Driving wires also contribute to fan-out delay.



Synchronous Circuits

Timing Analysis

NOR based SR Latch



```

LIBRARY ieee;
USE ieee.std_logic_1164.all;

-----

ENTITY SRlatch IS
    PORT (  s, r: IN std_logic;
           q, qbar: BUFFER std_logic);
END SRlatch;

-----

ARCHITECTURE dataflow OF SRlatch IS
BEGIN

    q <= not(r or qbar);
    qbar <= not(s or q);

END dataflow;
    
```

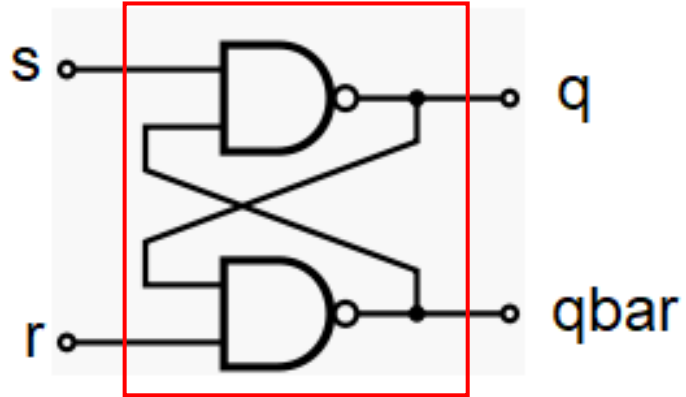
s	r	q	qbar
0	0	q	qbar
0	1	0	1
1	0	1	0
1	1	0	0

1-bit Memory=previous state (there are two possible different stable states)

Only a single stable state

Invalid input causes a race condition when the input changes at the same time from s=r='1' to s=r='0' (there are two possible different stable states)

NAND based SR Latch



```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

--
ENTITY SRlatch IS
    PORT ( s, r: IN std_logic;
          q, qbar: BUFFER std_logic);
END SRlatch;

--
ARCHITECTURE dataflow OF SRlatch IS
BEGIN

    q <= not (s and qbar);
    qbar <= not (r and q);

END dataflow;
```

s	r	q	qbar
0	0	1	1
0	1	1	0
1	0	0	1
1	1	q	qbar

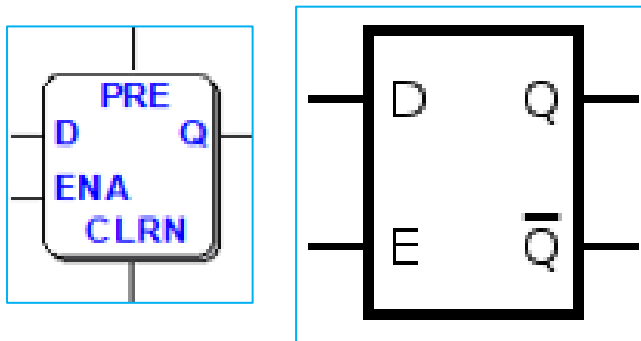
Invalid input causes a race condition when the input changes at the same time from $s=r=0$ to $s=r=1$ (there are two possible different stable states)

Only a single stable state

1-bit Memory=previous state (there are two possible different stable states)

Gated D-Latch

To eliminate the race condition (to achieve zero probability) in the *SR-Latch*, we need to modify it to a *Gated D-Latch*.

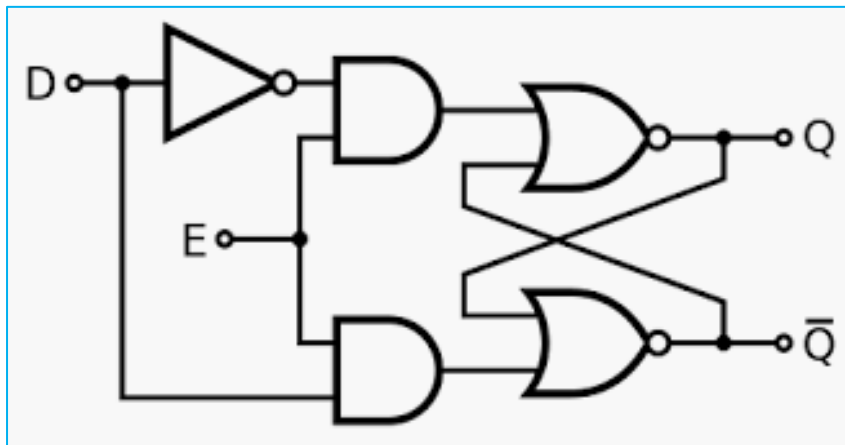


Symbol for a gated D latch

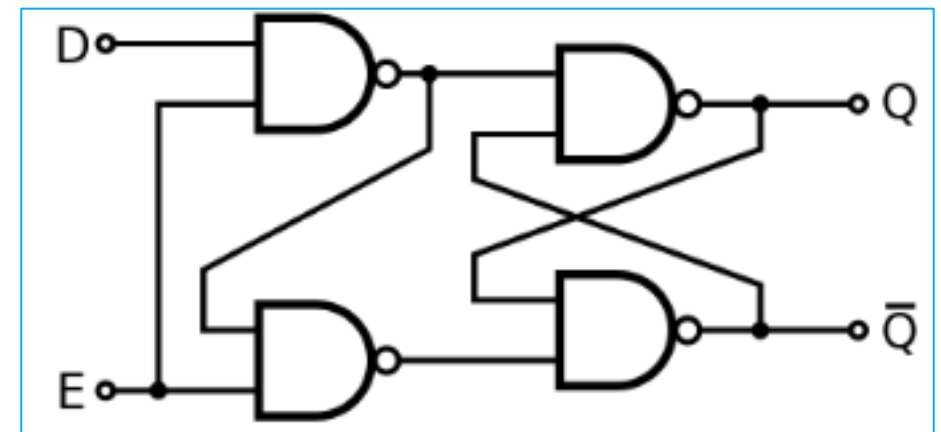
1-bit Memory=previous state

Write operation

		Output	
E=Enable	D=Data	Q	Qbar
0	x=don't care	Q	Qbar
1	0	0	1
1	1	1	0



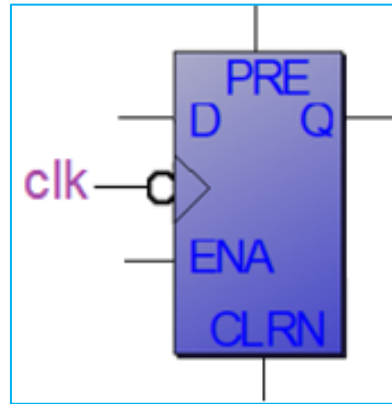
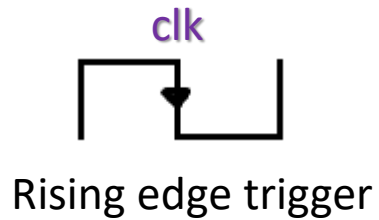
A gated D-latch based on an SR-latch NOR-based



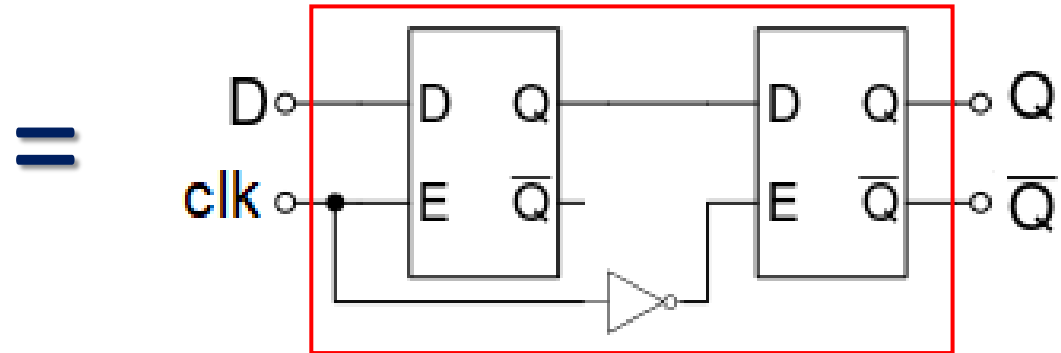
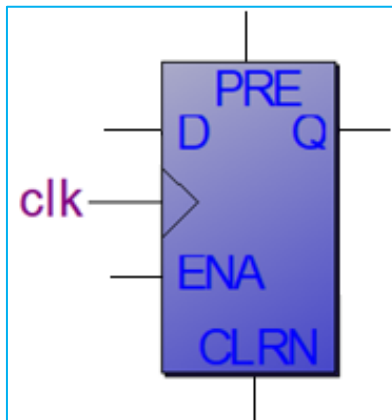
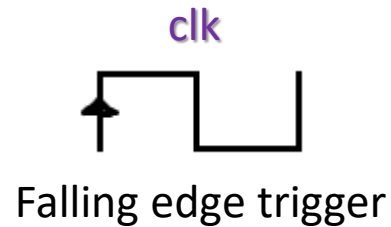
A gated D-latch based on an SR-latch NAND-based

D Flip-Flop = DFF

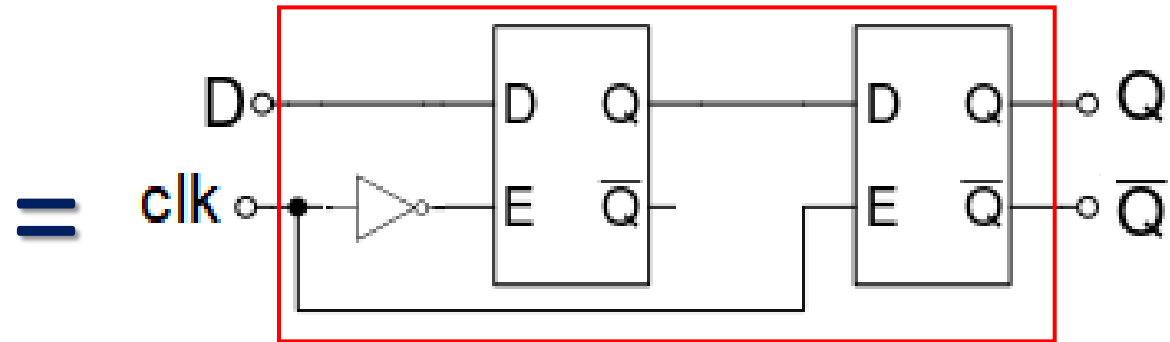
To sample the input data at a single point in time (positive or negative edge trigger), we need to use a rising/falling derivator structure.



Symbol for a DFF



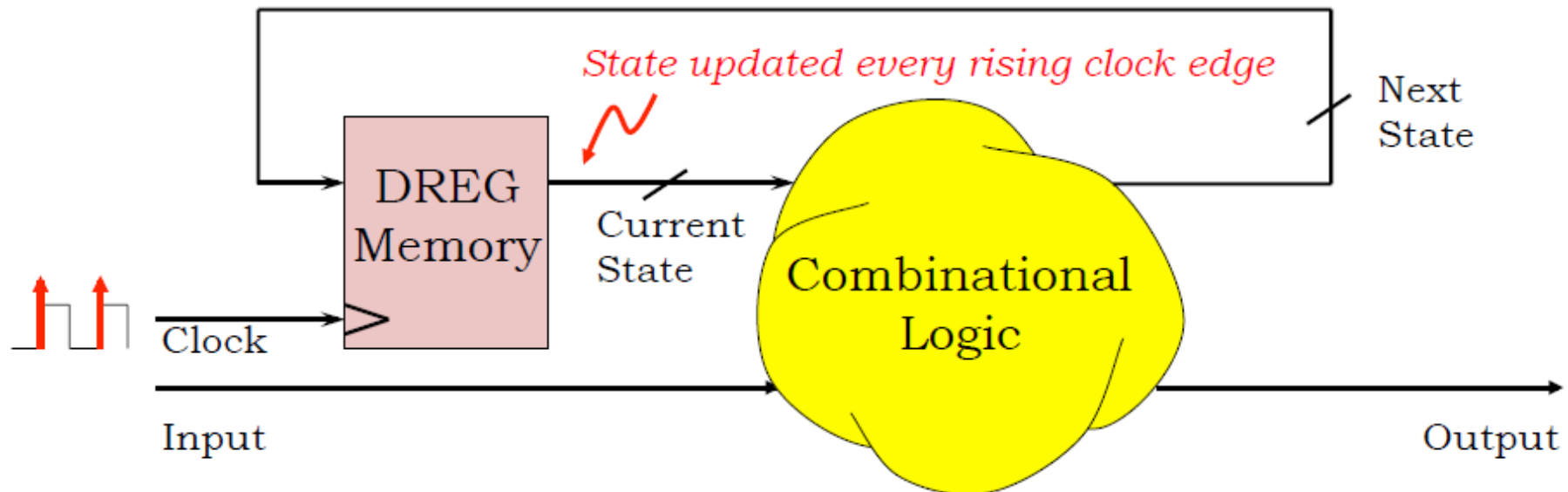
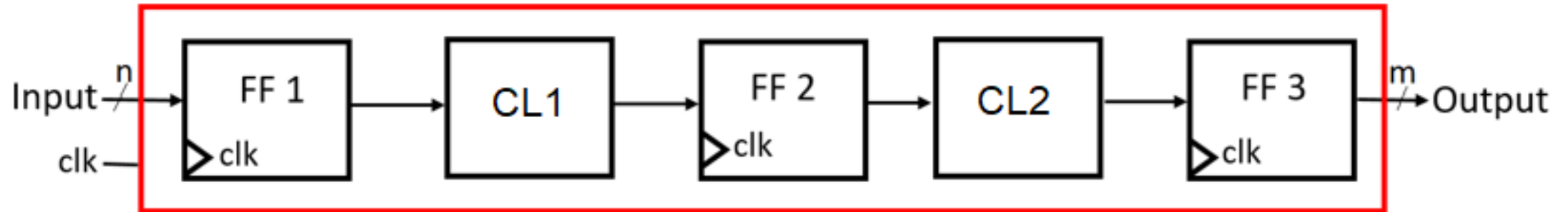
Negative edge DFF



Positive edge DFF

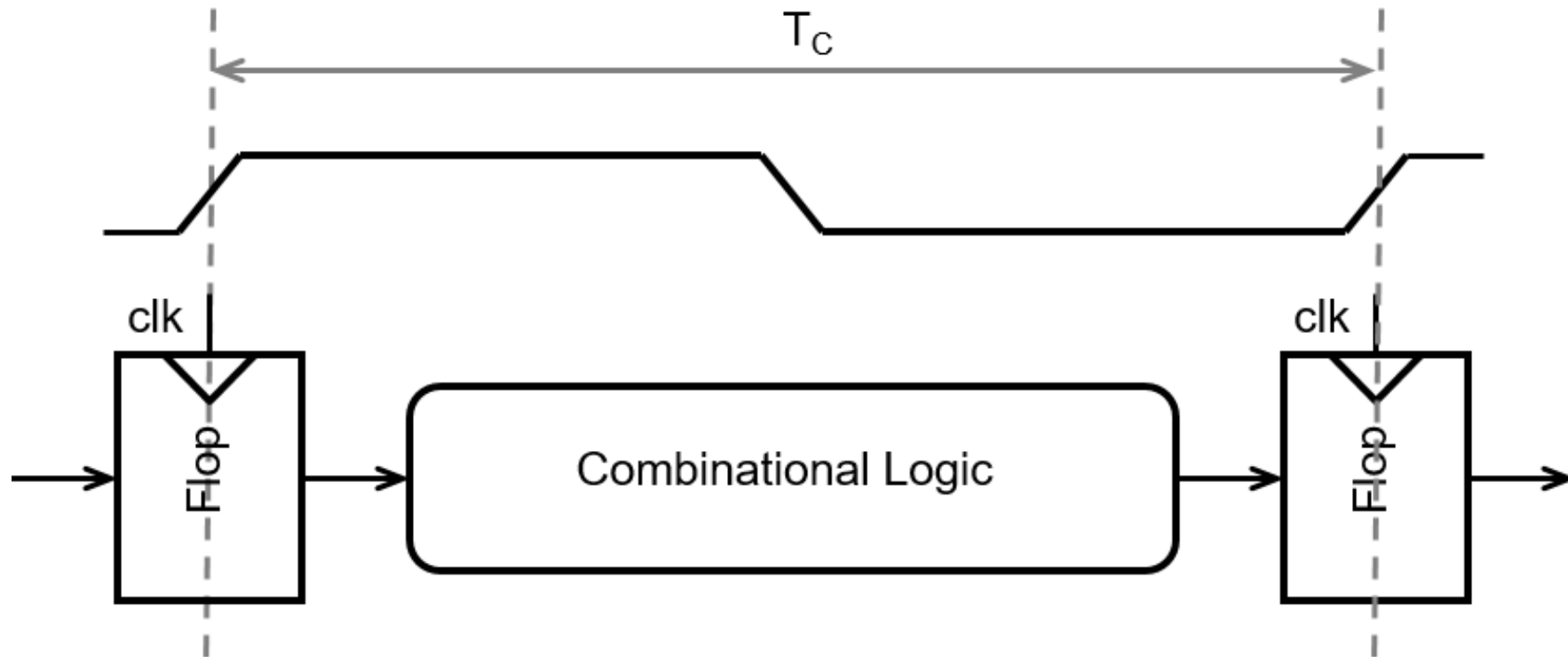
Dynamic Discipline Obedience

The output of each combinational logic path between two registers must be steady before each clock's transition (rising / falling edge).

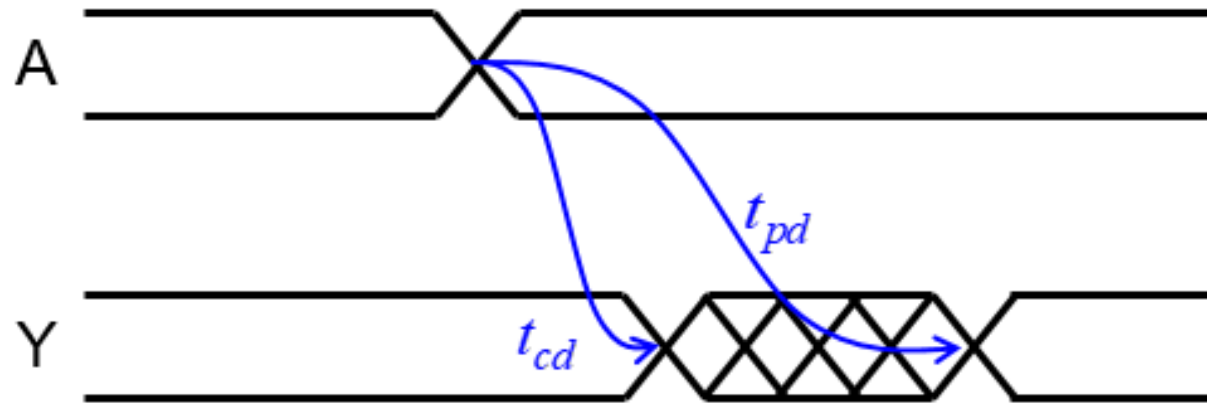
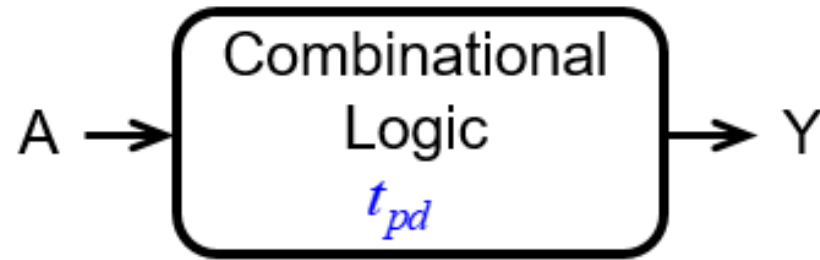


Synchronous Data Movement

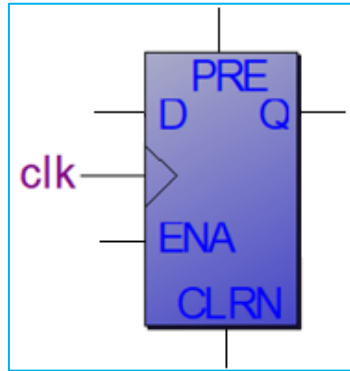
A single flip-flop is used on each cycle boundary. Data advances from one cycle to the next on each clock rising edge.



Reminder - Combinational timing analysis



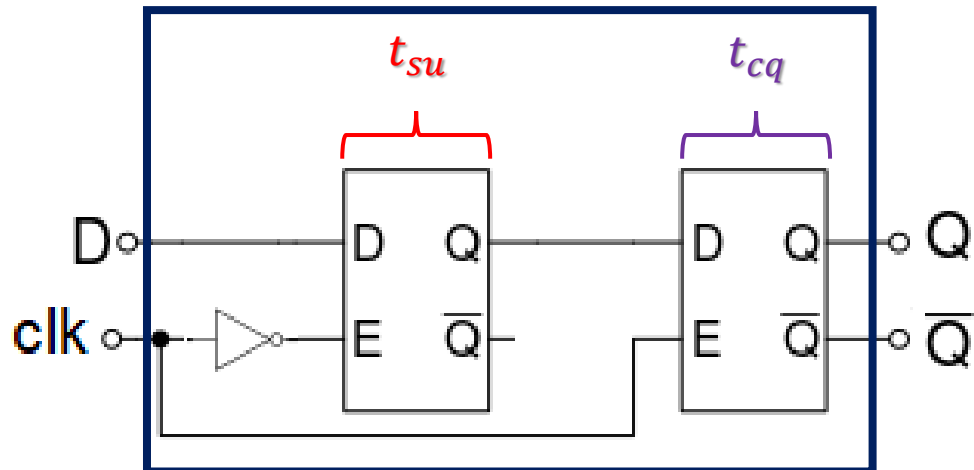
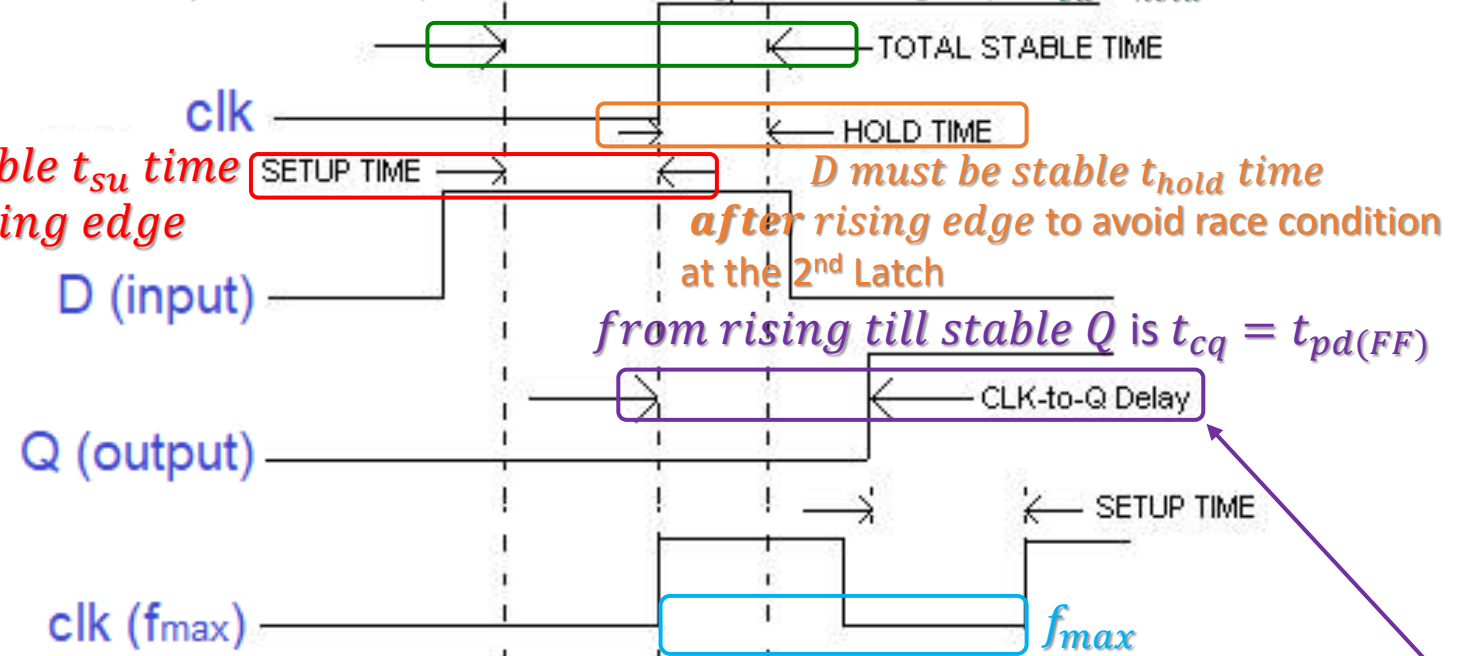
DFF timing analysis



Symbol for a DFF

D must be stable t_{su} time before rising edge

Aperture time (D must not change in this region) = $t_{su} + t_{hold}$



Positive edge DFF

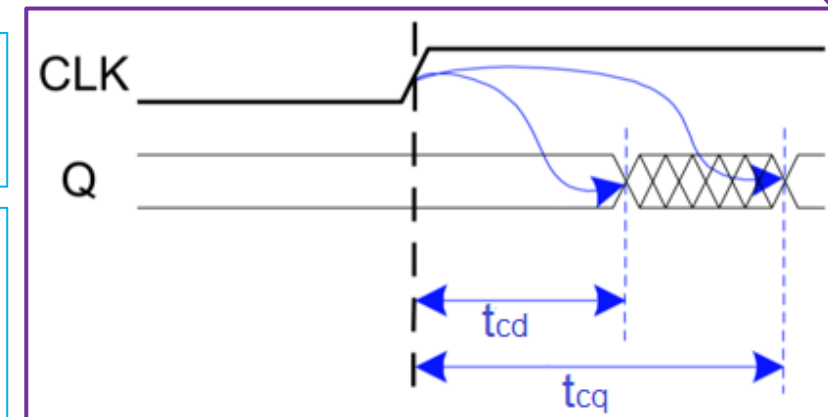


HW requirement:

$$t_{su} < t_{hold} < t_{cd} < t_{cq}$$

$$T_{min} = t_{cq} + t_{su}$$

$$f_{max} = \frac{1}{T_{min}} = \frac{1}{t_{cq} + t_{su}}$$



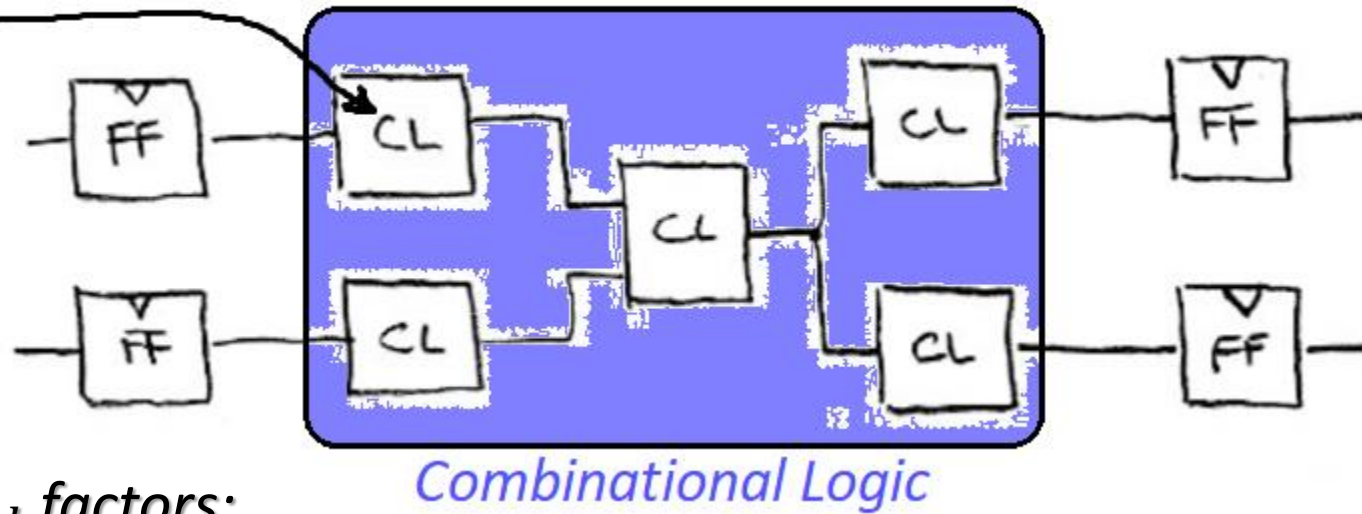
DFF Timing recap

- $t_{cq} = t_{pd(FF)}$: time after clock change that the output is guaranteed to be stable (propagation delay).
- t_{su} : time before clock edge, data must be stable (Setup time)
- t_{hold} : time after clock edge data must be stable (Hold time)
- $t_a = t_{su} + t_{hold}$: time around clock edge data must be stable (Aperture time)
- t_{cd} : time after clock edge that Q is stable by the previous value (Contamination delay)

Performance Measurement

Components of Path Delay

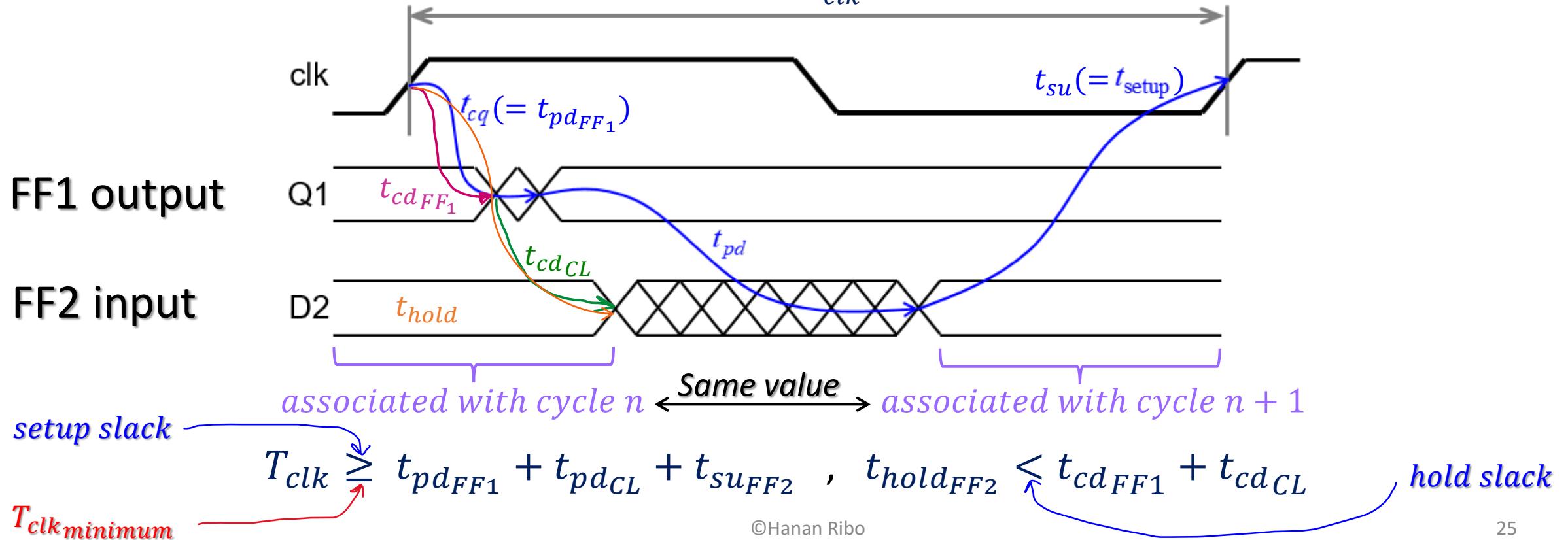
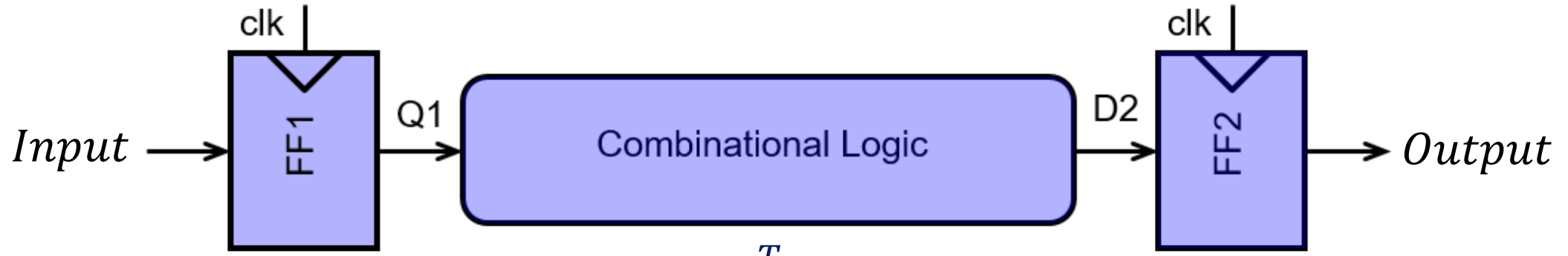
CL = Combinational
Logic Cell



Propagation delay t_{pd} factors:

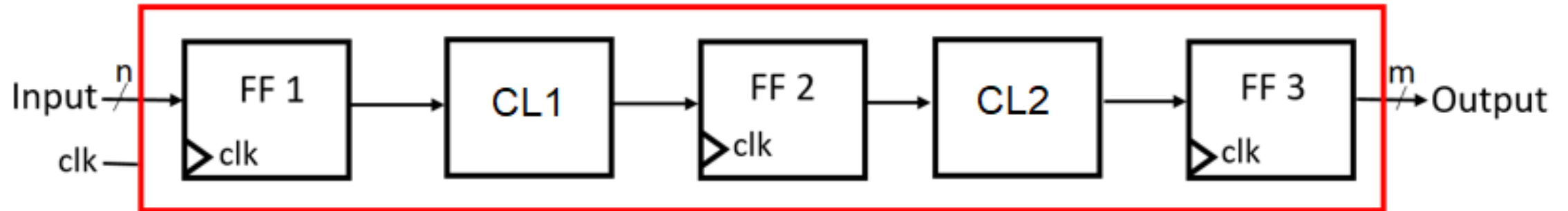
1. # of levels of logic
2. Internal cell delay
3. Wire delay
4. Cell input capacitance
5. Cell fan-out
6. Cell output drive strength

Timing Delay Constraints of t_{setup} and t_{hold}



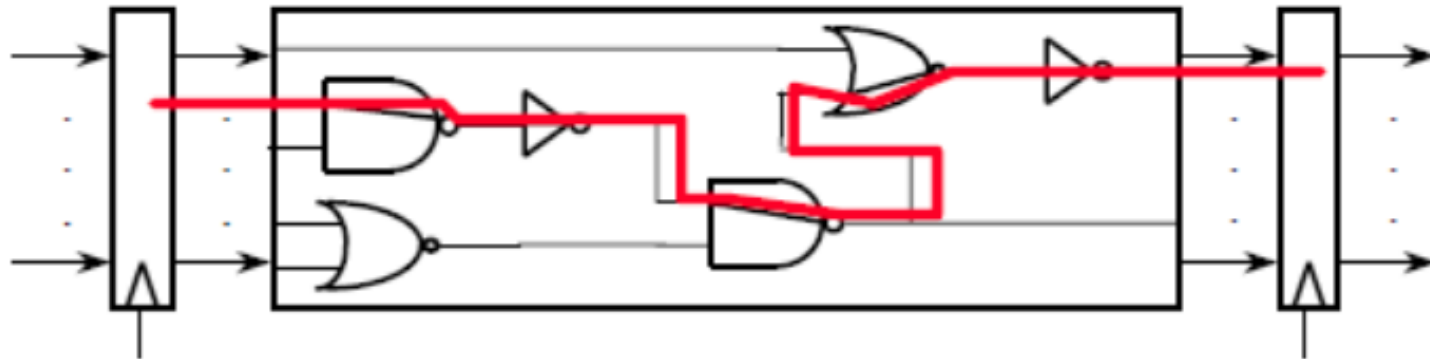
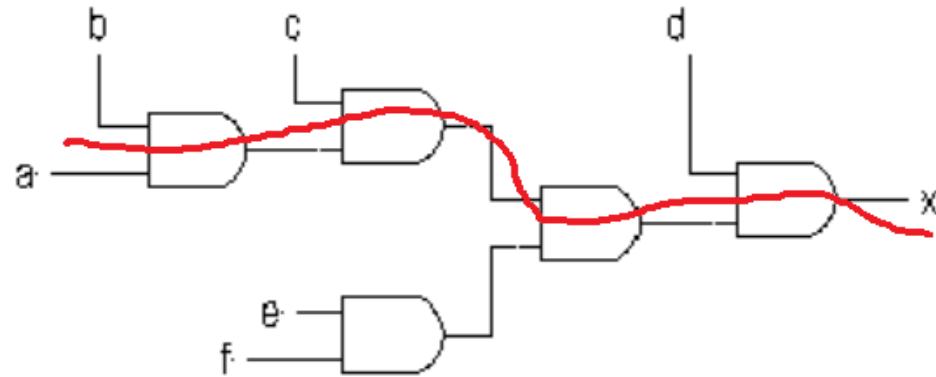
High Level System Description

- The most common mistake is to start HDL coding before describing the required digital system in high-level RTL form.
- In the RTL system description, it becomes clear what the system's combinational and synchronous subunits are.
- Any digital logic system can be disassembled to Combinational Logic blocks chained to FFs (registers).



Critical Path

- *Critical Path*: the path in the entire design with the maximum delay.
- This could be from state element to state element, from input to state element, state element to output, from input to output (unregistered paths).
- Example: what is the critical path in this circuit?



RTL design - example

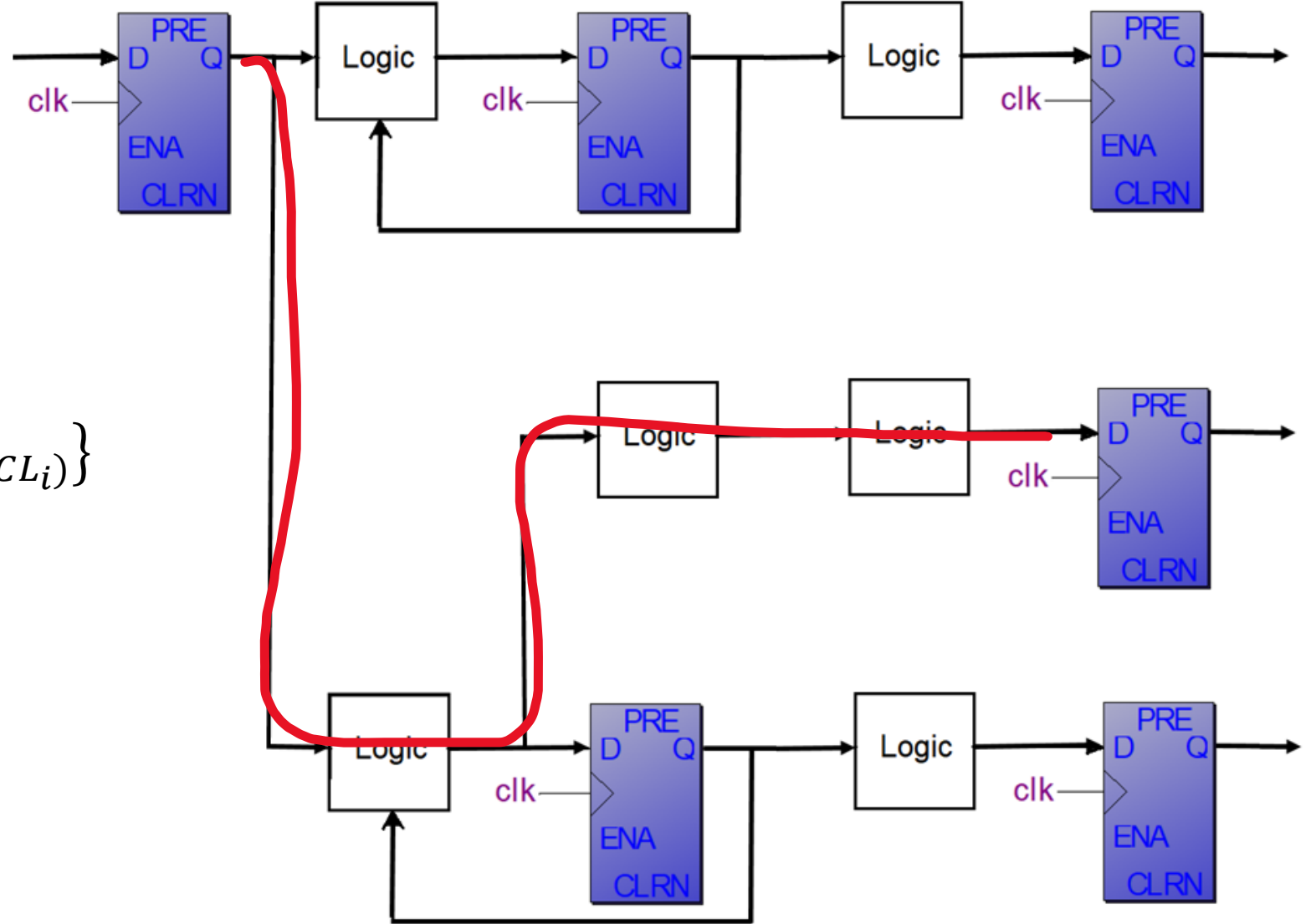
Question:

Where given that each logic cell has $t_{pd} = 1ns$, what is the critical path t_{pd} ?

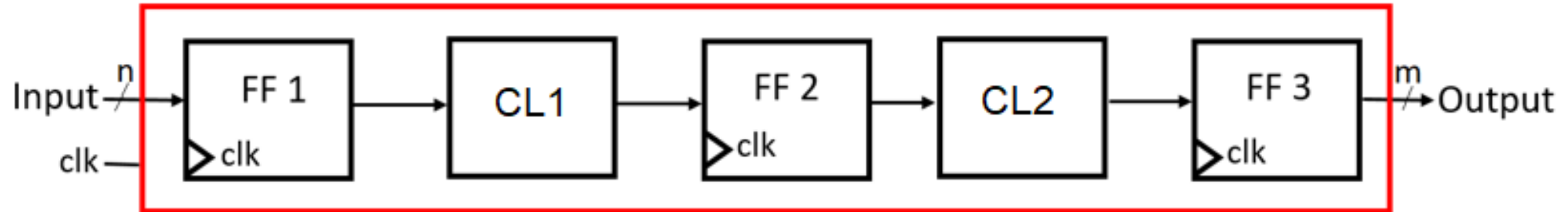
Answer:

critical path $t_{pd} = \max_i \{t_{pd}(CL_i)\}$

critical path $t_{pd} = 3ns$



f_{max} calculation – dynamic discipline Obedience



1. Notations:

- Each CL_i has its own $t_{pd}(CL_i)$
- We assume same technology for all FFs, means same $t_{cq}(= t_{pd_{FF}}), t_{cd}, t_{su}, t_{hold}$

2. Timing condition between the path from FF1 to FF2:

- FF2 setup condition: $t_{cq}(FF1) + t_{pd}(CL1) + t_{su}(FF2) \leq T_{clk}$
- FF2 hold condition: $t_{cd}(FF1) + t_{cd}(CL1) \geq t_{hold}(FF2)$

3. General expression for f_{max} : The Critical Path, we strive in our design for balanced CL paths

- Setup condition: $t_{cq} + \max_i \{t_{pd}(CL_i)\} + t_{su} = T_{min} \rightarrow f_{max} = \frac{1}{T_{min}}$
- Hold condition: $t_{cd} + \min_i \{t_{cd}(CL_i)\} \geq t_{hold}$

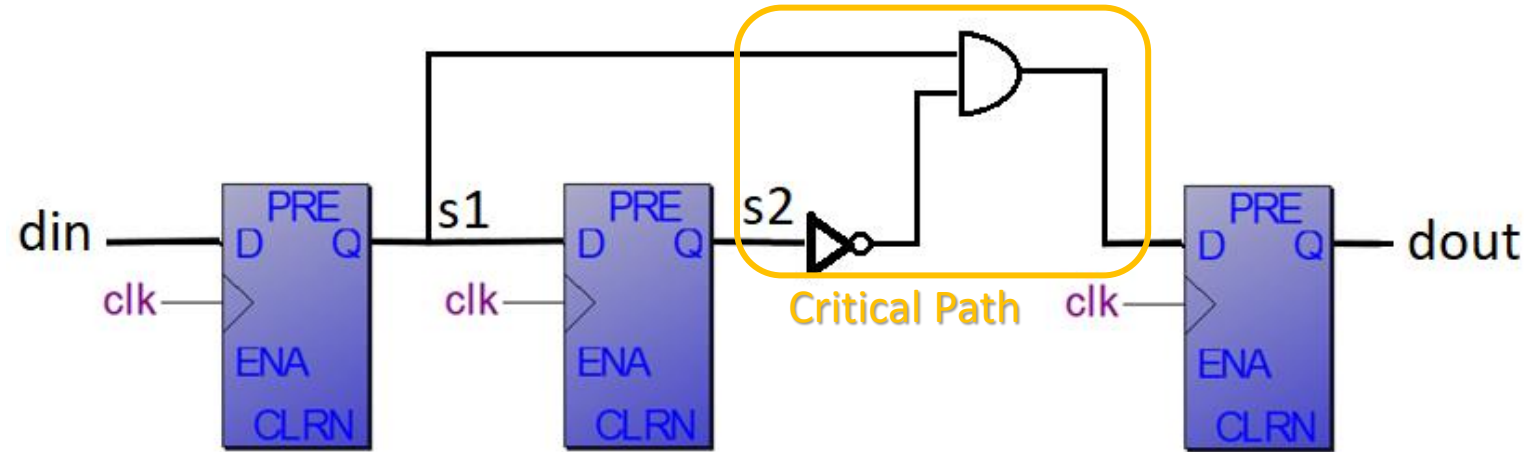
f_{max} calculation – example 1

The given data:

$$t_{su} = 200ps$$

$$t_{cq} = 300ps$$

$$t_{pd_{gate}} = 100ps$$



$$T_{min} = t_{cq} + 2 \cdot t_{pd_{gate}} + t_{su} = 700ps \rightarrow f_{max} = \frac{1}{T_{min}} = 1.428GHz$$

f_{max} calculation – example 2

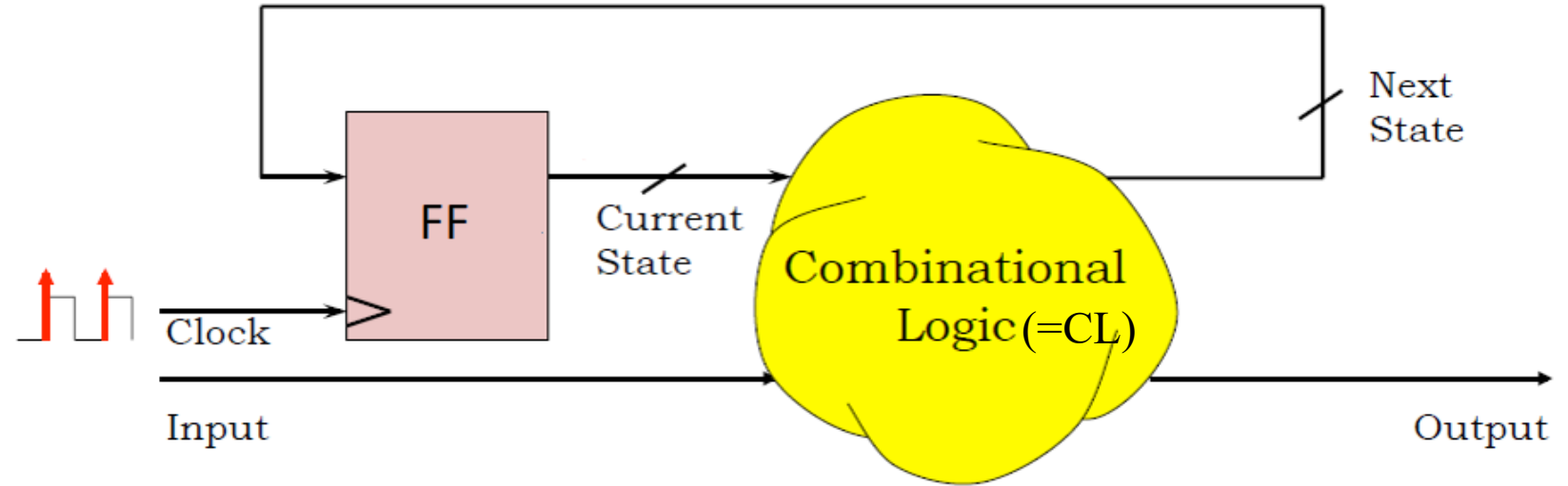
1. FF intrinsic delays:

$$t_{cd_{FF}} = 1ns, t_{pd_{FF}} = 3ns,$$

$$t_{su_{FF}} = 2ns, t_{hold_{FF}} = 2ns$$

2. CL delays:

$$t_{cd_{CL}} = ?, t_{pd_{CL}} = 5ns$$



Questions:

1. What is the constraint for the $t_{cd_{CL}}$?
2. What is the circuit f_{max} ?
3. In case $t_{cd_{CL}} = 2ns$ what is the slack value of t_{hold} ?

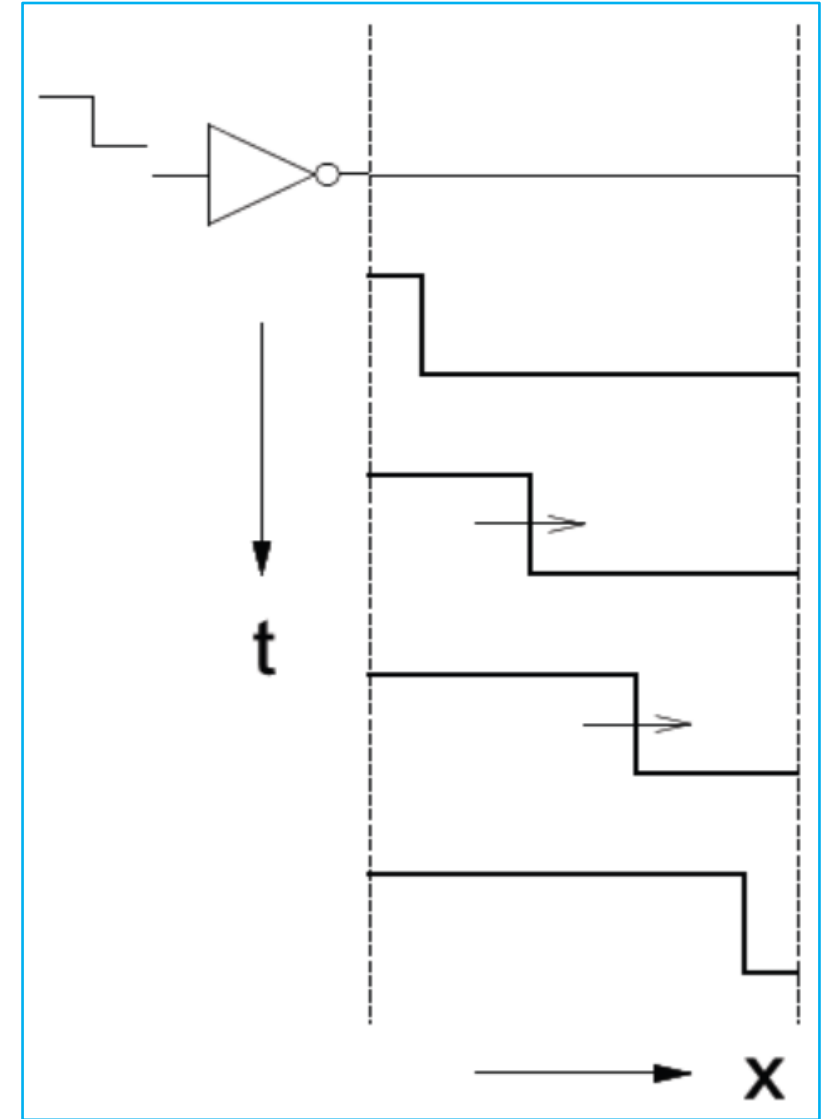
Answers:

1. The constraint: $t_{hold} \leq t_{cd_{FF}} + t_{cd_{CL}} \rightarrow 2ns \leq 1ns + t_{cd_{CL}} \rightarrow t_{cd_{CL}} \geq 1ns$
2. $T_{clk} \geq t_{pd_{FF}} + t_{pd_{CL}} + t_{su_{FF}} = 10ns \rightarrow f_{max} = \frac{1}{T_{clk}} = 100MHz$
3. $t_{hold_{FF}} \leq t_{cd_{FF}} + t_{cd_{CL}} \rightarrow slack_{hold} = t_{cd_{FF}} + t_{cd_{CL}} - t_{hold_{FF}} = 1ns$

Clock Skew *(Clock Uncertainty)*

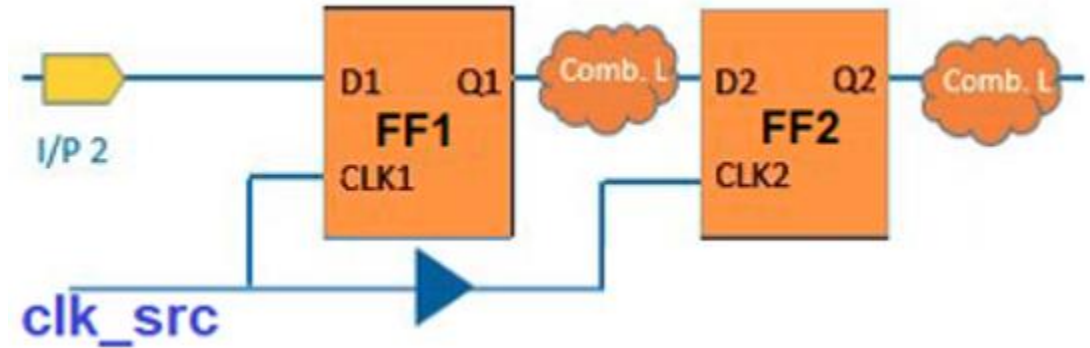
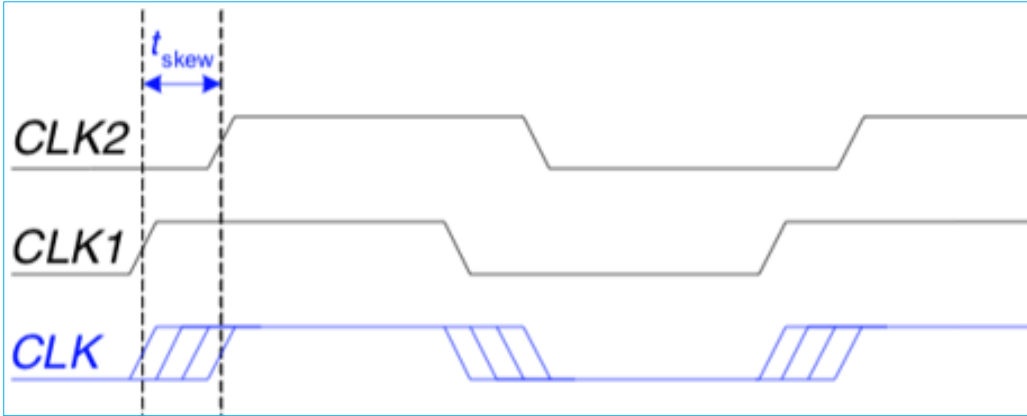
Wire Delay

- Signal wave-front moves close to the speed of light $\sim 1\text{foot/ns}$ (1 foot = 30.48cm)
- ICs' most wires are short, and the transit times are relatively short (can be ignored) compared to the clock period, *except for clock lines (longer than ICs' most wires, the clock signals take time to move from one location to another).*
- Physically, the wires have *resistance*, *capacitance*, and *inductance (frequency-dependent)*, causing clock skew (phase difference) between the systems' FFs.



Clock Skew

Clock should theoretically arrive simultaneously at all sequential circuits. Practically, it arrives at different times. The differences are called **clock skews**.

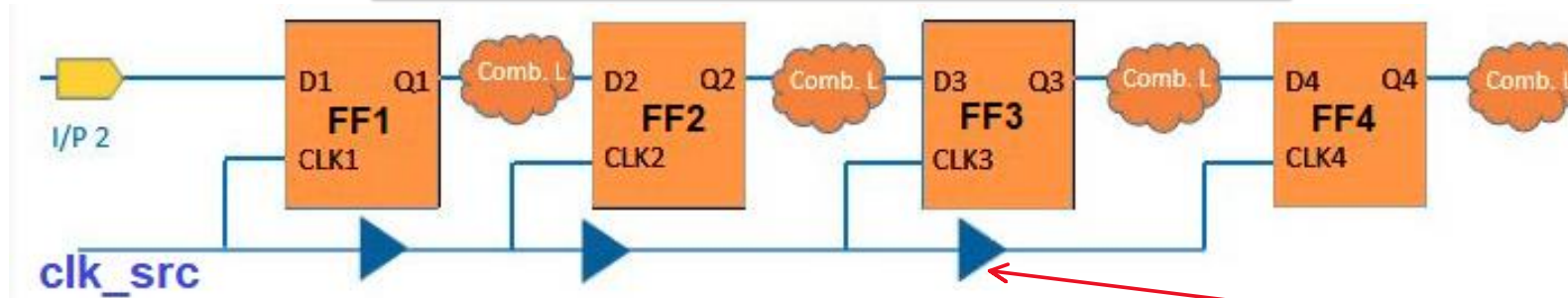


$$t_{skew} \triangleq CLK(PHASE)_{capture_{FF}} - CLK(PHASE)_{launch_{FF}} = CLK2(PHASE) - CLK1(PHASE)$$

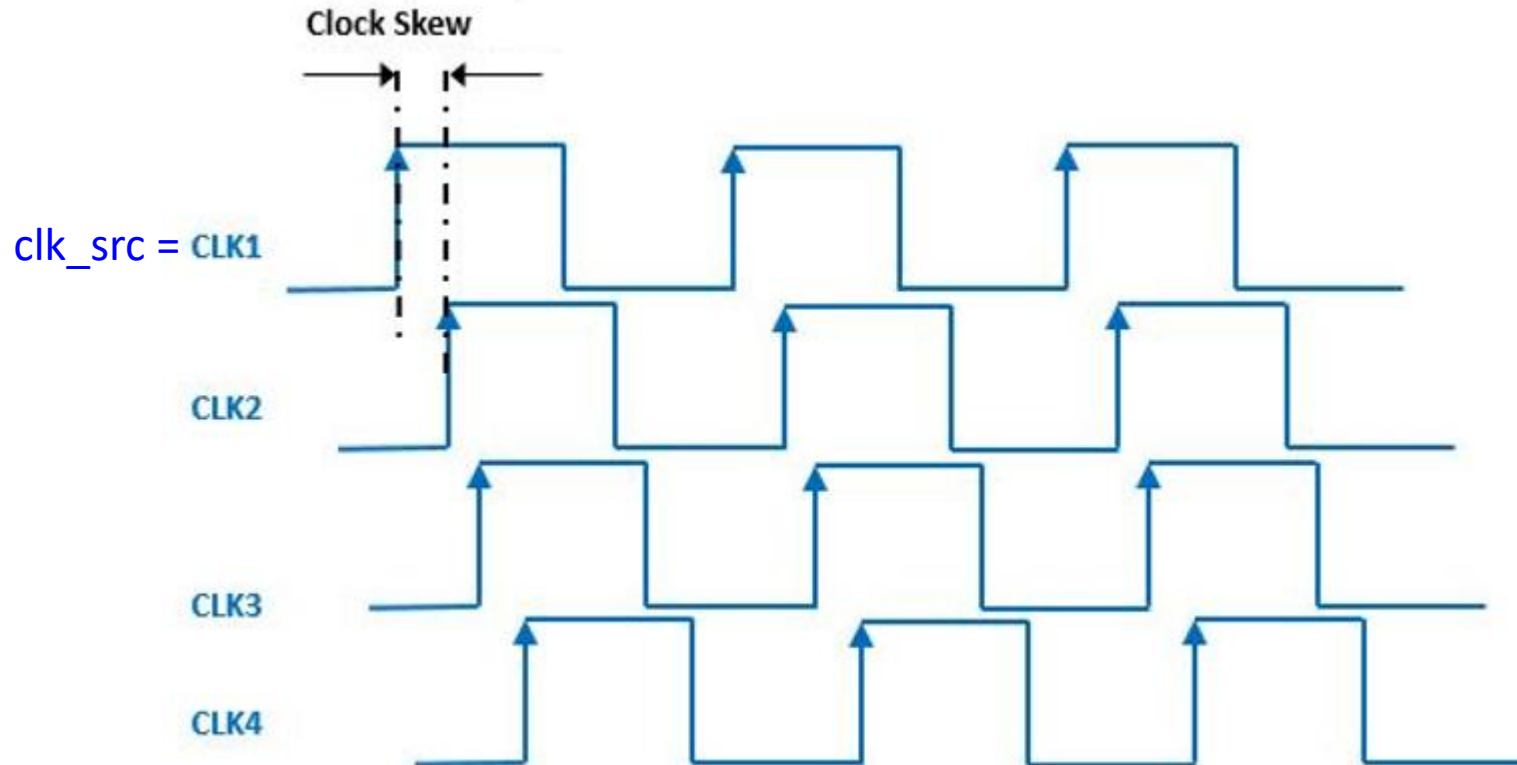
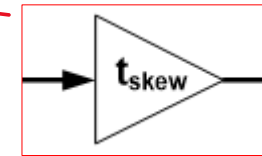
Clock skew consists of the following components:

- **Systematic** is the portion existing under nominal conditions. It can be minimized by an appropriate design.
- **Random** is caused by process variations like devices' channel length, oxide thickness, threshold voltage, wire thickness, width, and space. It can be measured on silicon and adjusted by delay components.

Positive Clock Skew



$$CLK(PHASE)_{capture_{FF}} > CLK(PHASE)_{launch_{FF}} \rightarrow T_{skew} > 0$$

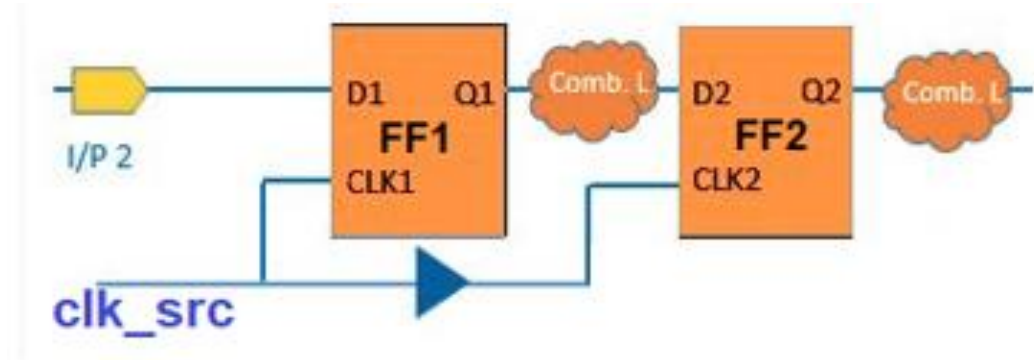
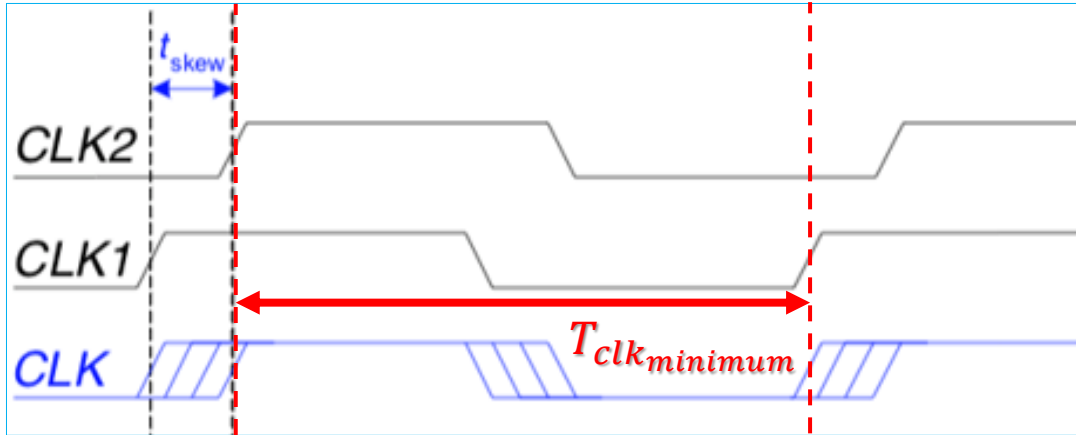


Explanation:

The clock arrives at the capture FFs **later** than it does at the launch FF. This effectively extends the time available for the data propagation, allowing a shorter clock period (a higher frequency).

Positive Clock Skew

To guarantee that dynamic discipline is not violated for any register, we perform the worst-case analysis.

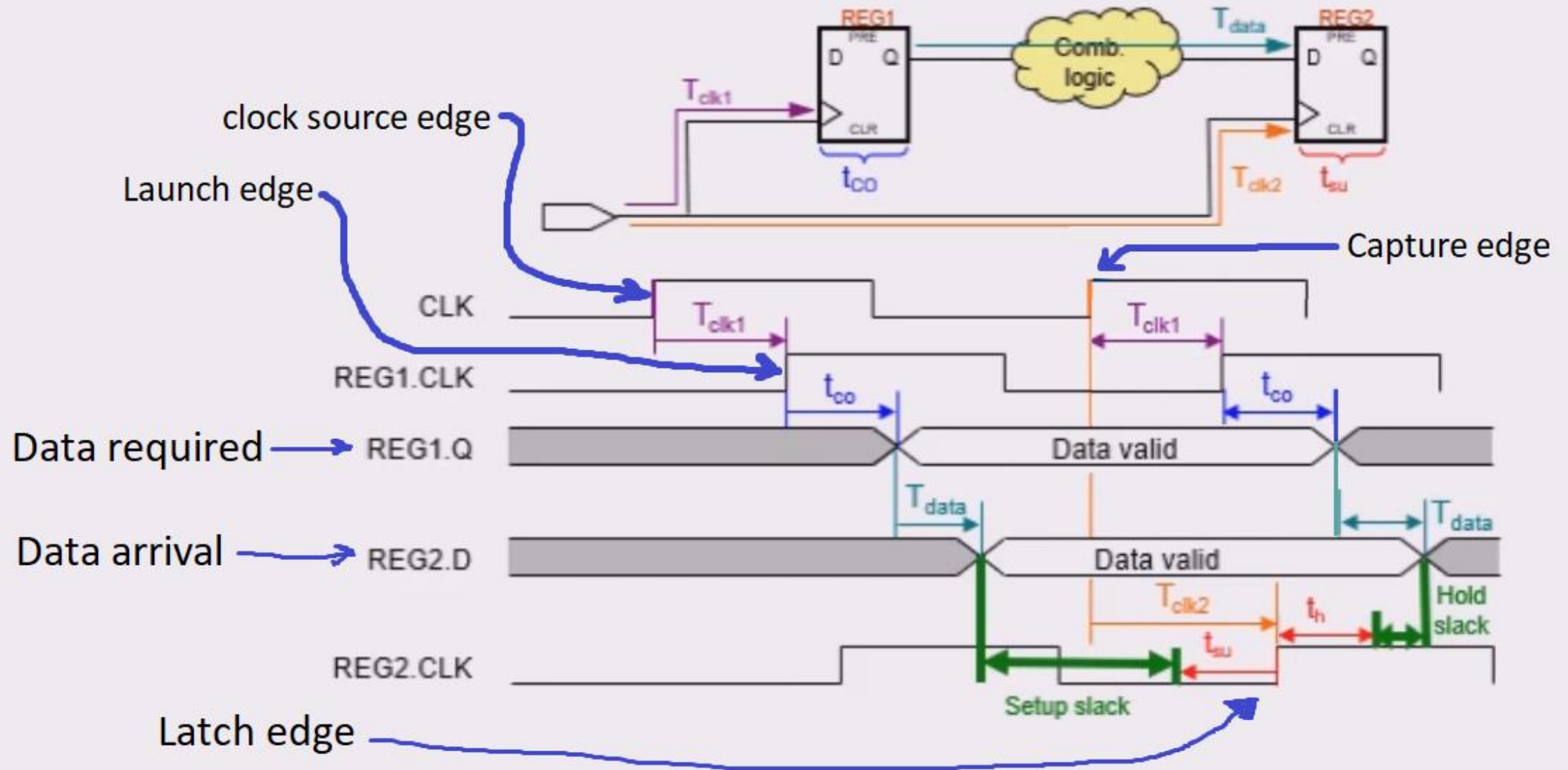


$$clk2(phase) = clk1(phase) + t_{skew} \rightarrow t_{skew} > 0$$

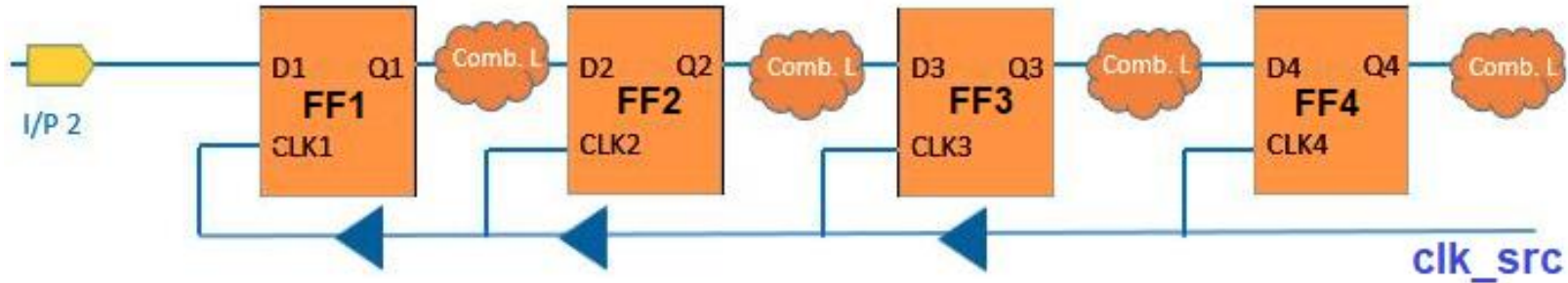
Timing constraint (FF1 to FF2):

1. $T_{clk} \geq t_{pd_{FF1}} + t_{pd_{CL1}} + t_{su_{FF2}} - t_{skew} \rightarrow \text{higher } f_{max}$
2. $t_{hold_{FF2}} < t_{cd_{FF1}} + t_{pd_{CL1}} - t_{skew} \rightarrow \text{reduced hold margin (slack)}$
3. $t_{su_{FF2}} \rightarrow \text{increased setup margin (slack)}$

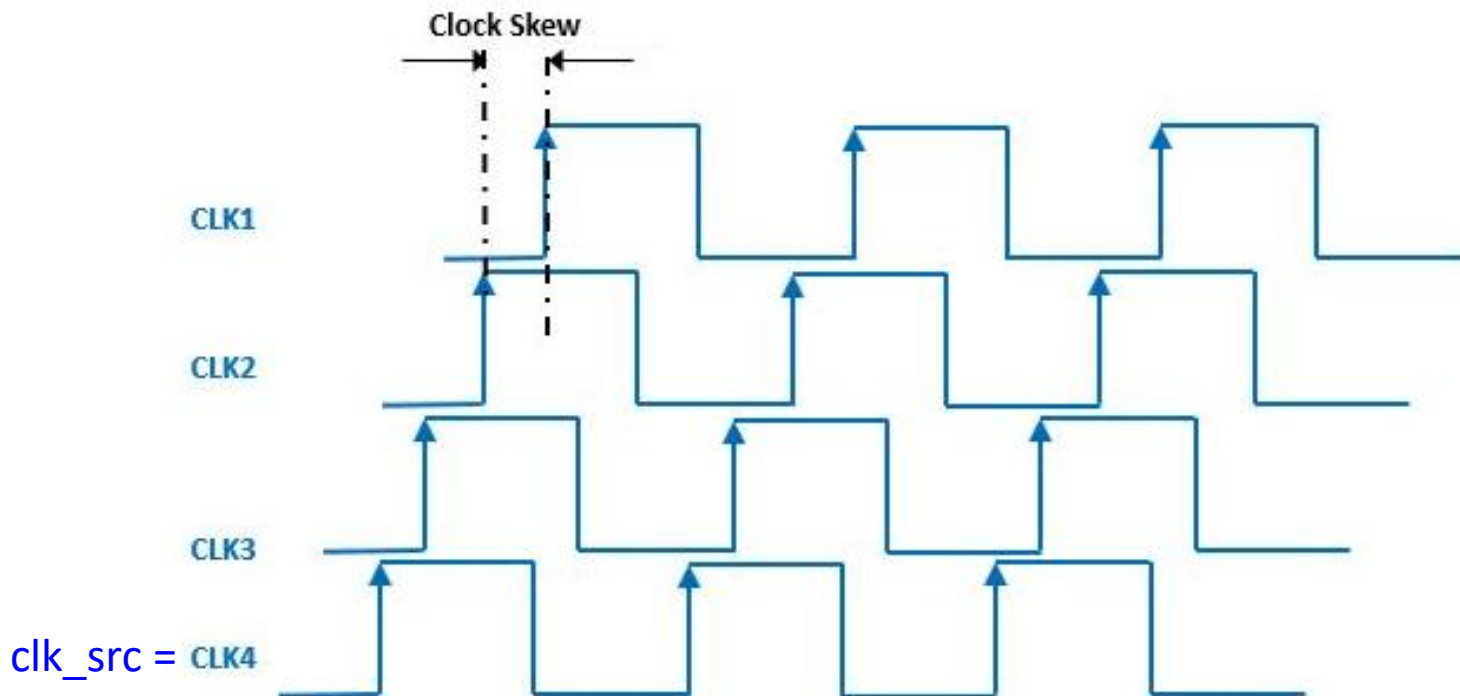
Positive Clock Skew



Negative Clock Skew



$$CLK(PHASE)_{capture_{FF}} < CLK(PHASE)_{launch_{FF}} \rightarrow T_{skew} < 0$$



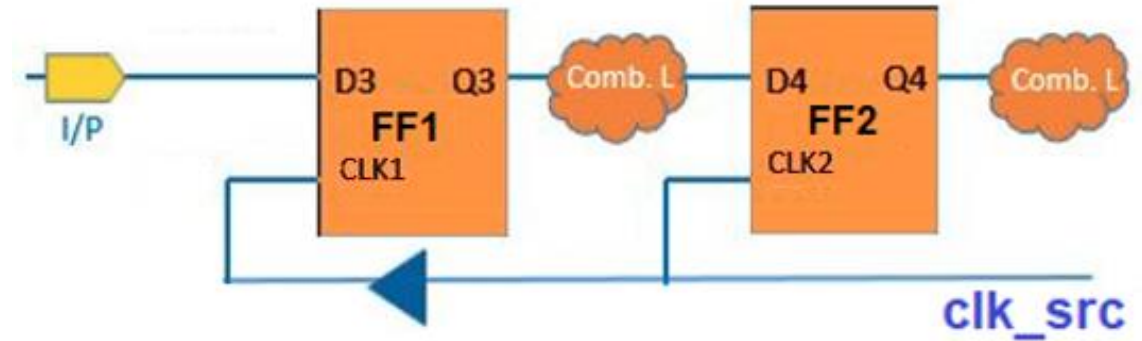
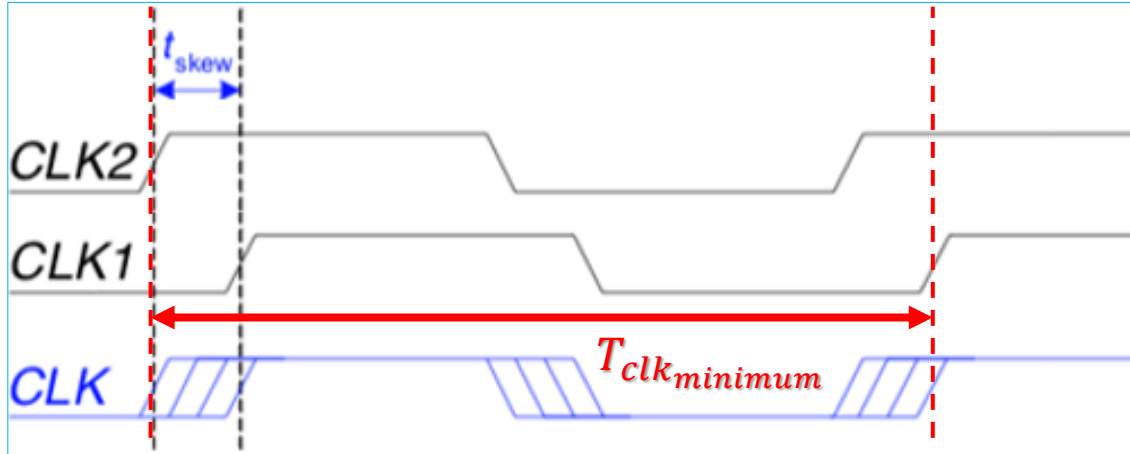
Explanation:

The clock arrives at the capture FFs *earlier* than the launch FF.

This reduces the time available for data propagation, requiring a longer clock period to ensure the data is stable before the clock edge.

Negative Clock Skew

To guarantee that dynamic discipline is not violated for any register, we perform the worst-case analysis.

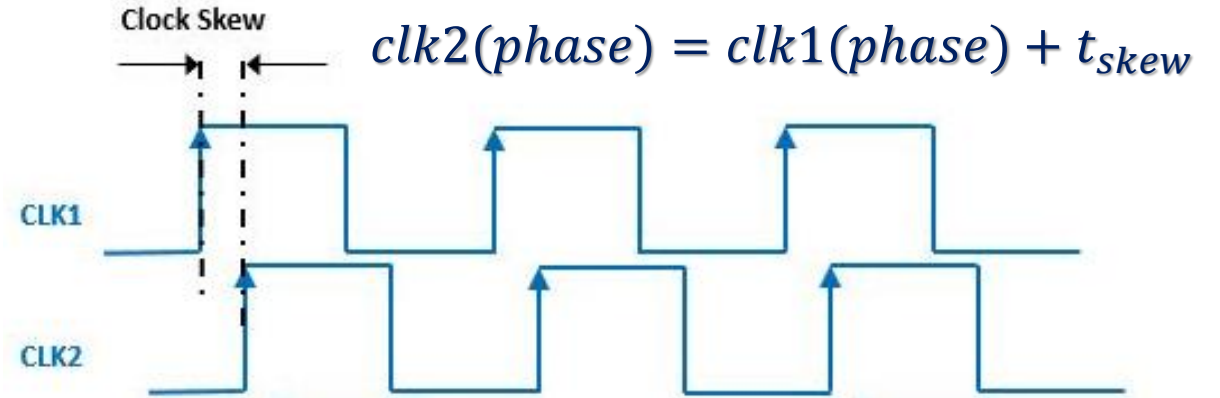
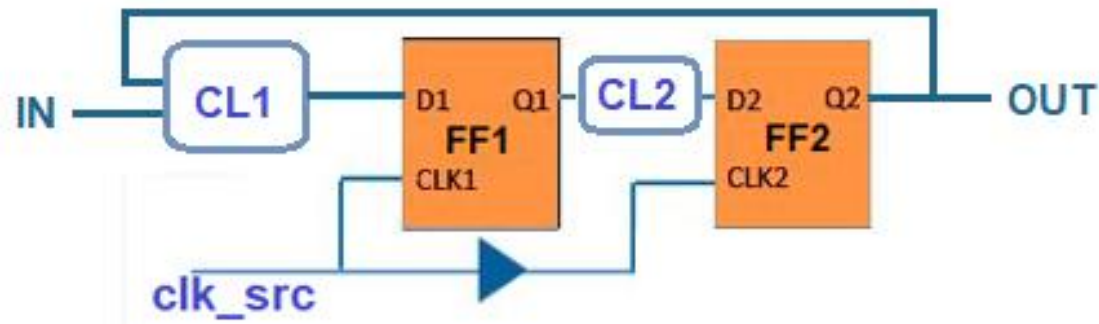


$$clk2(phase) = clk1(phase) + t_{skew} \rightarrow t_{skew} < 0$$

Timing constraint (FF1 to FF2):

1. $T_{clk} \geq t_{pd_{FF1}} + t_{pd_{CL1}} + t_{su_{FF2}} + |t_{skew}| \rightarrow \text{lower } f_{max}$
2. $t_{hold_{D2}} < t_{cd_{FF1}} + t_{pd_{CL1}} + |t_{skew}| \rightarrow \text{increased hold margin (slack)}$
3. $t_{su_{D2}} \rightarrow \text{reduced setup margin}$

Clock Skew - example



Data: $t_{su} = 2ns$, $t_{hold} = 3ns$, $t_{cd} = 4ns$, $t_{cq} = 5ns$, $t_{pd_{CL1}} = 4ns$, $t_{pd_{CL2}} = 2ns$

Question:

What is the system f_{max}

Answer:

FF1 to FF2 (positive skew):

$$t_{cd} + t_{pd_{CL2}} - t_{hold} \geq (\text{phase}_{FF2} - \text{phase}_{FF1}) \rightarrow t_{skew} \leq 3ns$$

$$T_{min1} = t_{cq} + t_{pd_{CL2}} + t_{su} - (\text{phase}_{FF2} - \text{phase}_{FF1}) = 5ns + 2ns + 2ns - 3ns = 11ns$$

FF2 to FF1 (negative skew):

$$t_{cd} + t_{pd_{CL1}} - t_{hold} \geq (\text{phase}_{FF1} - \text{phase}_{FF2}) \rightarrow t_{skew} \geq -5ns$$

$$T_{min2} = t_{cq} + t_{pd_{CL1}} + t_{su} - (\text{phase}_{FF1} - \text{phase}_{FF2}) = 5ns + 4ns + 2ns - (-5ns) = 16ns$$

$$T_{min} = \max(T_{min1}, T_{min2}) = 16ns$$

$$f_{max} = 1/T_{min} = 62.5MHz$$